

FIAP ON

10

NLP E CHATBOTS: VOZES DIGITAIS QUE COMPREENDEM

LISTA DE FIGURAS

Figura 1 - Linha do tempo das tecnologias NLP.....	25
Figura 2 - Exemplo de compreensão diferente de expressão idiomática (diferença cultural) com o Microsoft Copilot gerando imagens por prompt de texto.....	28
Figura 3 - Esquemático para categorizar algoritmos de NLP de acordo com o tipo de entrada e saída e a capacidade de entendimento e geração de texto	37
Figura 4 - Diagrama de Venn das categorias de algoritmos de NLP de acordo com a capacidade de recuperar ou extrair informação	38
Figura 5 - Diagrama de Venn com a categorização de Chomsky das gramáticas formais	41
Figura 6 - Sequência de telas para o download do corpus de texto do Projeto Gutenberg	57
Figura 7 - Demonstração de Lei de Zipf em Quincas Borba de Machado de Assis. Esse mesmo gráfico é gerado ao ser executado o código-fonte anterior.....	59

LISTA DE QUADROS

Quadro 1 - Comparação entre sistemas de Recuperação de Informação	30
---	----

EMENDAS

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo básicos de Regex em Python	46
Código-fonte 2 - Extração de URLs usando Regex em Python.....	48
Código-fonte 3 - Código Python para o download do modelo de tokenização do NLTK	52
Código-fonte 4 - Demonstração do uso do analisador léxico do NLTK em Python ...	52
Código-fonte 5 - Remoção de stopwords usando NLTK em Python	53
Código-fonte 6 - Demonstração do uso do analisador léxico do spaCy em Python ..	55
Código-fonte 7 - Remoção de stopwords usando spaCy em Python	55
Código-fonte 8 - Análise de frequência dos tokens presentes em Quincas Borba....	58

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Instalação do NLTK	52
Comando de prompt 2 - Instalação do spaCy	54
Comando de prompt 3 - Instalação do módulo de português do spaCy.....	54

EMPRE

SUMÁRIO

1 NLP E CHATBOTS: VOZES DIGITAIS QUE COMPREENDEM	8
1.1 O que é linguagem natural?	8
1.2 O que é Processamento de Linguagem Natural?	9
1.3 Qual é a história das tecnologias de NLP?	10
1.3.1 ELIZA	10
1.3.2 PARRY	11
1.3.3 SHRDLU	11
1.3.4 RACTER	12
1.3.5 INTELLECT	12
1.3.6 Candide	13
1.3.7 A.L.I.C.E.	13
1.3.8 Office Assistant Clippit	14
1.3.9 Jabberwacky e Cleverbot	14
1.3.10 Google Translate	15
1.3.11 Word2Vec	15
1.3.12 IBM Watson	16
1.3.13 Cortana	16
1.3.14 Siri	17
1.3.15 Alexa	17
1.3.16 Google Assistant	17
1.3.17 Microsoft Tay	18
1.3.18 Watson Assistant	18
1.3.19 Google DialogFlow	19
1.3.20 Rasa	19
1.3.21 BERT	19
1.3.22 RoBERTa	20
1.3.23 T5	20
1.3.24 GPT-2 e GPT-3	20
1.3.25 BART	21
1.3.26 ALBERT	21
1.3.27 DistilBERT	22
1.3.28 GPT-3.5 e MetaAI	22
1.3.29 Copilot	23
1.3.30 Gemini	23
1.3.31 ChatGPT-4	23
1.3.32 DeepSeek	24
2 APLICAÇÕES DE NLP	26
2.1 Corretores Ortográficos	26
2.2 Tradutores Automáticos	26
2.3 Recuperação de Informação	28
2.4 Sumarização de Texto	30
2.5 Análise de Sentimento	31
2.6 Resposta a Perguntas, Chatbots, Assistentes Virtuais e Agentes	32
3 CATEGORIZAÇÃO DOS ALGORITMOS DE NLP	35
3.1 Sistemas de Transformação	35
3.2 Entendimento e Geração	36

3.3 Recuperação e Extração de Informação	37
4 GRAMÁTICAS FORMAIS E EXPRESSÕES REGULARES.....	39
4.1 Hierarquia de Chomsky	39
4.2 Expressões regulares.....	42
5 ANÁLISE LEXICAL	49
5.1 Analisadores Léxicos em Compiladores.....	49
5.2 Analisadores Léxicos em NLP.....	50
5.3 Lei de Zipf.....	56
REFERÊNCIAS.....	60

UNIVERSIDADE

1 NLP E CHATBOTS: VOZES DIGITAIS QUE COMPREENDEM

1.1 O que é linguagem natural?

Por **linguagem natural** nos referimos a línguas que surgiram naturalmente ao longo da história humana, como o português, o inglês, o japonês e o espanhol, entre diversos outros idiomas. Linguagens naturais possuem sons (fonemas) e símbolos gráficos (grafemas) que carregam significado (semântica) e seguem regras de construção (sintaxe). Podemos dizer que uma determinada língua (idioma) é uma instância de uma linguagem natural possível.

Por outro lado, com o avanço dos estudos linguísticos, alguns padrões entre diferentes línguas foram descobertos, de maneira que línguas artificiais puderam ser propostas com base nesses padrões. Alguns exemplos de línguas construídas são o Esperanto e o Klingon. Essas línguas não surgiram espontaneamente ao longo do tempo, mas foram criadas de forma deliberada por indivíduos ou grupos, geralmente com objetivos específicos em mente, como a comunicação internacional (no caso do Esperanto) ou a criação de universos fictícios (como o Klingon, de Star Trek).

As **línguas artificiais** podem ser projetadas para ter características específicas, como uma gramática regular e sem ambiguidades, ou para atender a necessidades específicas de comunicação em contextos particulares. Enquanto as línguas naturais evoluem ao decorrer do tempo, influenciadas por fatores culturais, históricos e sociais, as línguas artificiais podem ser mais sistemáticas e controladas, buscando clareza e precisão.

As **linguagens de programação** podem ser entendidas como linguagens artificiais, cujo objetivo é permitir uma interface entre o programador (aquele que escreve na linguagem de programação) e o computador. O estudo das linguagens de programação é realizado pela área de Paradigmas de Programação, que define quais são os tipos de linguagem, como procedural, orientada a objetos, funcional, lógica, entre outras, e os princípios que orientam a construção de cada uma delas. Essas linguagens possuem regras e sintaxes próprias, assim como as línguas naturais, mas com um foco específico: possibilitar que um ser humano se comunique de forma eficiente com uma máquina para criar programas de computador. O estudo dos paradigmas de programação também se estende à análise da evolução das

linguagens de programação, identificando como as novas linguagens surgem para resolver limitações das anteriores e como elas se adaptam às necessidades da computação moderna, como concorrência, processamento paralelo e programação distribuída.

O nome dado às linguagens de programação em contraste com as linguagens naturais é **linguagens formais**. Linguagens formais são aquelas que possuem uma gramática rigorosa e bem definida, com regras específicas sobre a sintaxe e semântica, sem flexibilidade ou ambiguidades, como ocorre nas línguas naturais. Essas regras determinam exatamente como os símbolos podem ser organizados para formar expressões válidas dentro da linguagem. Enquanto nas línguas naturais (como o português, o inglês etc.), a comunicação pode ser mais flexível — com uma gramática mais aberta e sujeita a variações e interpretações contextuais —, as linguagens de programação exigem uma estrutura exata e sem espaço para ambiguidade. Se o programador cometer um erro na sintaxe ou na estrutura do código, o compilador ou interpretador geralmente resulta em um erro de execução.

1.2 O que é Processamento de Linguagem Natural?

Da mesma forma que podemos escrever um código de computador para ler uma linguagem formal como Python ou C++ (programas estes chamados de interpretadores e compiladores), também podemos escrever um código de computador que “interpreta um programa escrito em linguagem humana”. Como programa, neste caso, nos referimos a um texto genérico, que pode conter instruções detalhadas ou conhecimento acerca de um domínio. Assim, a área de **Processamento de Linguagem Natural** (PLN) — conhecida em inglês pela sigla NLP (Natural Language Processing), a qual iremos adotar ao longo desse texto — surgiu a partir da interseção entre Linguística, Inteligência Artificial (IA) e Ciência da Computação, com o intuito de permitir que as máquinas entendam, interpretem e gerem a linguagem humana de forma útil.

Historicamente, muitos dos problemas e desafios fundamentais do Processamento de Linguagem Natural foram inicialmente explorados no contexto da análise de linguagens formais e compiladores. A partir da década de 1950, a Ciência da Computação começou a aplicar técnicas de gramáticas formais, como as

gramáticas regulares e livres de contexto, para entender e estruturar linguagens formais — inicialmente usadas para descrever linguagens de programação. Isso levou ao desenvolvimento de ferramentas como analisadores léxicos (lexers) e analisadores sintáticos (parsers), essenciais para compilar código e estruturar suas relações. Esses mesmos conceitos foram, posteriormente, aplicados ao NLP para entender a estrutura da linguagem natural — um domínio muito mais complexo devido à ambiguidade, à variação e às nuances do discurso humano. Tais ferramentas, como a tokenização e o parsing, começaram a ser adaptadas para lidar com a análise de texto, originando técnicas que permitiram que as máquinas processassem e interpretassem a linguagem humana com mais precisão. Além disso, o uso de expressões regulares, amplamente desenvolvido para verificar padrões em linguagens formais, tornou-se um método essencial no NLP para tarefas como extração de informações e reconhecimento de padrões em grandes volumes de texto.

1.3 Qual é a história das tecnologias de NLP?

Nos anos 1950, a ideia de máquinas “entenderem” a linguagem humana foi estimulada pelo desenvolvimento inicial da Inteligência Artificial. Um dos primeiros marcos foi o trabalho de Alan Turing, que, em 1950, propôs o famoso **Teste de Turing**, uma avaliação para determinar se uma máquina é capaz de exibir comportamento inteligente equivalente ao de um ser humano, utilizando como critério a capacidade de se comunicar por linguagem natural.

1.3.1 ELIZA

Nas décadas de 1960 e 1970, os primeiros sistemas de Processamento de Linguagem Natural eram construídos com base em abordagens simbólicas, ou seja, utilizavam conjuntos de regras linguísticas definidas manualmente por especialistas. Esses sistemas dependiam fortemente de gramáticas formais, que descreviam a estrutura da língua com precisão, e de dicionários elaborados manualmente, que associavam palavras a seus significados e funções gramaticais. Um dos exemplos mais emblemáticos dessa época foi o **ELIZA**, criado por Joseph Weizenbaum em 1966. ELIZA simulava um psicoterapeuta rogeriano que reproduzia padrões de conversa por meio de regras simples de associação de palavras-chave e respostas

pré-definidas. Apesar de seu funcionamento bastante superficial, ELIZA conseguiu enganar alguns usuários, levando-os a acreditar que estavam interagindo com um ser inteligente — um feito notável para a época e que chamou bastante a atenção da comunidade científica e do público em geral.

1.3.2 PARRY

Outro exemplo importante foi o **PARRY**, desenvolvido por Kenneth Colby em 1972. Enquanto ELIZA simulava um terapeuta, PARRY simulava um paciente com esquizofrenia paranoide. A diferença entre os dois estava no nível de complexidade: PARRY incorporava um modelo rudimentar de estados mentais e crenças, tentando imitar o comportamento de um paciente real, incluindo respostas emocionais e crenças irracionais. Quando ELIZA e PARRY foram colocados para conversar entre si, a interação gerou tanto fascínio quanto reflexão sobre os limites desses sistemas.

1.3.3 SHRDLU

Depois de ELIZA e PARRY, foi criado outro programa notável dentro da abordagem simbólica: o projeto **SHRDLU**, desenvolvido por Terry Winograd entre 1968 e 1970. O sistema permitia que usuários interagissem com um mundo virtual — uma espécie de “micromundo” composto por blocos e comandos —, utilizando linguagem natural. Embora operasse em um domínio extremamente restrito e controlado, o SHRDLU demonstrou um grau mais avançado de compreensão linguística em comparação com ELIZA e PARRY. Enquanto esses dois últimos se baseavam em respostas padronizadas e associações superficiais entre palavras-chave, SHRDLU utilizava uma representação lógica do ambiente e das ações possíveis, permitindo inferências e diálogos mais coerentes dentro de seu universo limitado. Essa diferença não apenas evidenciava o potencial das abordagens simbólicas quando aliadas a um conhecimento estruturado do mundo, mas também deixava claro o desafio de escalar esses sistemas para contextos mais amplos e imprevisíveis. Apesar de não ter sido desenvolvido com esse propósito, SHRDLU acabou se tornando um dos primeiros jogos de ficção interativa. Nele, a narrativa era predominantemente apresentada em forma escrita, permitindo que jogadores ou

leitores inserissem comandos de texto para controlar personagens do jogo e influenciar o ambiente.

1.3.4 RACTER

Durante os anos 1980, surgiram experimentos que exploravam o uso da linguagem natural de maneira criativa e provocativa. Um exemplo notável foi o **Racter**, um programa de computador lançado em 1983, que ganhou notoriedade por ter “escrito” o livro *The Policeman’s Beard is Half Constructed*. Racter utilizava regras gramaticais e estruturas pré-definidas para gerar frases que soavam surreais e poéticas, muitas vezes sem sentido lógico, mas que despertavam interesse por sua aparente criatividade. Embora estivesse longe de compreender a linguagem no sentido humano, o Racter evidenciava como algoritmos podiam manipular a linguagem para gerar textos com características estilísticas, abrindo caminho para usos literários e artísticos do NLP.

1.3.5 INTELLECT

Outro sistema influente da mesma época foi o **INTELLECT**, um software lançado pela Symantec em meados dos anos 1980, que funcionava como uma interface de linguagem natural para bancos de dados. O INTELLECT permitia que usuários realizassem consultas complexas usando uma linguagem quase natural, como *“List all employees hired before 1980 and earning more than \$40,000”*. Seu diferencial estava em traduzir essas instruções em comandos SQL, tornando o acesso à informação mais intuitivo para usuários leigos. Apesar de suas limitações em entender ambiguidades e contextos sutis, o INTELLECT foi um precursor de tecnologias que, atualmente, estão presentes em assistentes corporativos e sistemas de BI com interface conversacional.

A partir da década de 1980, a área de NLP começou a transitar gradualmente das abordagens simbólicas para métodos estatísticos, impulsionada pelo crescimento da capacidade computacional e pela disponibilidade de grandes volumes de dados linguísticos digitalizados. Esse movimento marcou uma nova era na pesquisa em NLP, na qual a ênfase passou a recair sobre a análise empírica de grandes corpora textuais,

permitindo o treinamento de modelos com base em probabilidades extraídas do uso real da linguagem.

Um marco importante desse período foi o desenvolvimento de **modelos n-grama**, usados amplamente na década de 1990. Esses modelos estatísticos baseavam-se na ideia de prever a próxima palavra em uma sequência com base nas palavras anteriores. Por exemplo, um modelo trigrama (3-gram) estima a probabilidade de uma palavra ocorrer com base nas duas anteriores. Embora simples, esses modelos se mostraram bastante úteis em tarefas que envolviam a correção ortográfica, a tradução automática e o reconhecimento de fala, tornando-se a espinha dorsal de muitos sistemas de NLP por quase duas décadas.

1.3.6 Candide

A década de 1990 também foi marcada por avanços em **Machine Translation** (MT), com destaque para o sistema **Candide**, da IBM, que aplicava técnicas estatísticas para traduzir automaticamente entre idiomas. A Statistical Machine Translation (SMT) substituiu os sistemas baseados em regras com uma abordagem baseada em grandes corpora bilíngues, como o Europarl (corpus de traduções do Parlamento Europeu), permitindo que os sistemas aprendessem padrões de tradução diretamente dos dados.

1.3.7 A.L.I.C.E.

Na virada para os anos 2000, com o acesso crescente à internet e a evolução dos chatbots, surgiu **A.L.I.C.E.** (Artificial Linguistic Internet Computer Entity), desenvolvida por Richard Wallace em 1995. ALICE foi um dos chatbots mais sofisticados de sua geração e três vezes vencedor do **Prêmio Loebner**, uma competição baseada no Teste de Turing. Construído com base na linguagem de marcação AIML (Artificial Intelligence Markup Language), o sistema utilizava padrões de entrada e resposta para simular conversas. Apesar de ainda depender fortemente de scripts pré-programados, ALICE demonstrava como estruturas de diálogo mais elaboradas poderiam dar a impressão de uma conversa fluida, influenciando o desenvolvimento posterior de assistentes virtuais.

1.3.8 Office Assistant Clippit

Em paralelo aos avanços dos chatbots, os editores de texto também começaram a incorporar funcionalidades inteligentes, influenciadas por técnicas de NLP. Ferramentas como corretores ortográficos, sugestões de palavras e análise gramatical automática tornaram-se padrão em softwares como o **Microsoft Word**. Esses recursos, que hoje consideramos básicos, foram frutos do avanço de algoritmos estatísticos e heurísticos capazes de analisar grandes volumes de texto e sugerir correções contextualmente apropriadas. Eles representaram um dos primeiros contatos do público geral com aplicações práticas de NLP no dia a dia.

Nesse contexto, merece destaque o **Office Assistant Clippit**, mais conhecido como “Clippy”, o famoso assistente animado da Microsoft, introduzido em 1997. Clippy foi um esforço ambicioso de integrar um agente interativo com capacidades rudimentares de NLP no ambiente de produtividade do Microsoft Office. Embora tenha se tornado um ícone da cultura pop — frequentemente lembrado com certo humor ou irritação —, Clippy tentava prever as necessidades do usuário com base em suas ações e sugerir dicas úteis em linguagem natural. Apesar de suas limitações, ele foi um precursor dos assistentes inteligentes modernos, como Cortana, Siri e Google Assistant, demonstrando o potencial (e os desafios) de tornar a linguagem natural parte da interação homem-máquina em larga escala.

Nos anos 2000, os avanços em **Aprendizado de Máquina Supervisionado** ganharam destaque. Algoritmos como Máquinas de Vetor de Suporte (SVM) e Árvores de Decisão foram amplamente utilizados em tarefas de classificação de texto, como análise de sentimentos, categorização de e-mails (spam *versus* não spam) e extração de informações. Além disso, houve avanços significativos no desenvolvimento de **etiquetadores gramaticais** e **parsers estatísticos**, que aumentaram a precisão na identificação de categorias gramaticais e estruturas sintáticas das frases.

1.3.9 Jabberwacky e Cleverbot

O **Jabberwacky**, criado por Rollo Carpenter em 1988 e disponibilizado online em 1997, foi um projeto pioneiro de chatbot que se destacava por manter conversas mais naturais e fluidas, utilizando **Aprendizado Contínuo** para melhorar suas respostas ao longo do tempo. Ao contrário de chatbots anteriores, como ELIZA ou

PARRY, que seguiam regras fixas, o Jabberwacky aprendia com as interações em tempo real, o que o tornava mais adaptável, embora suas respostas ainda fossem imprevisíveis. Em 2008, Carpenter lançou o **Cleverbot**, uma evolução do Jabberwacky. Utilizando um banco de dados muito maior de conversas, o Cleverbot aprimorou o aprendizado de máquina, oferecendo respostas mais precisas. Disponível em várias plataformas móveis, ampliou seu alcance e se tornou uma ferramenta ainda mais popular.

1.3.10 Google Translate

O **Google Translate**, lançado em 2006, representou um marco significativo no campo de NLP, ao popularizar o uso de Modelos Estatísticos e, posteriormente, Redes Neurais Profundas para **tradução automática**. Inicialmente, o serviço utilizava um modelo de tradução baseado em phrase-based translation, que dividia as frases em unidades menores e traduzia essas unidades de forma independente. Isso resultou em traduções que, embora funcionais, frequentemente careciam de fluidez e precisão. O verdadeiro avanço ocorreu com a introdução do Neural Machine Translation (NMT) em 2016, que utiliza Redes Neurais Profundas para traduzir textos de forma contextualizada, modelando frases inteiras em vez de pedaços isolados. Através de modelos como o Transformer, o Google Translate conseguiu melhorar significativamente a qualidade das traduções, com maior compreensão de contexto e redução de erros gramaticais. O uso de grandes corpora multilíngues também permitiu que o sistema se beneficiasse de um Aprendizado Supervisionado eficaz, ajustando-se de maneira mais precisa às nuances de cada idioma. Esse avanço não só aumentou a precisão, mas também democratizou o acesso a traduções instantâneas, afetando desde a comunicação pessoal até o comércio internacional e o turismo.

1.3.11 Word2Vec

A grande virada no NLP veio na década de 2010, com o surgimento das técnicas de **Aprendizado Profundo (Deep Learning)**. Um dos primeiros e mais influentes modelos dessa nova era foi o **Word2Vec**, desenvolvido por Tomas Mikolov no Google em 2013. Esse modelo introduziu o conceito de **vetores de palavras (word embeddings)**, que representam palavras como pontos em um espaço vetorial

contínuo, capturando relações semânticas com base no contexto em que aparecem. Isso permitiu uma compreensão mais rica da linguagem, com capacidades como identificar analogias (por exemplo, "rei" - "homem" + "mulher" $\approx$ "rainha").

Durante esse período, diversas aplicações de NLP começaram a ganhar destaque, transformando a interação entre seres humanos e máquinas e impactando áreas como atendimento ao cliente, assistência pessoal, saúde e muito mais. Exemplos notáveis incluem IBM Watson, Cortana, Alexa, Siri e Google Assistant.

1.3.12 IBM Watson

O **IBM Watson** foi um dos pioneiros na aplicação de NLP em contextos empresariais e acadêmicos. Lançado com grande visibilidade após sua vitória no programa de TV "Jeopardy!" em 2011, o Watson demonstrou a capacidade de processar e entender grandes volumes de dados textuais, utilizando técnicas de NLP e Aprendizado de Máquina. O Watson foi aplicado em áreas como diagnóstico médico, atendimento ao cliente e até mesmo no setor financeiro, ajudando empresas a automatizar processos complexos e tomar decisões mais informadas. Seu poder de compreender perguntas em linguagem natural e fornecer respostas precisas e contextualizadas foi um marco importante no desenvolvimento de sistemas inteligentes.

1.3.13 Cortana

A **Cortana**, assistente virtual da Microsoft, também foi um exemplo notável de aplicação de NLP durante esse período. Desenvolvida a partir de 2009 e lançada em 2014, a Cortana utilizava tecnologias de NLP para entender comandos de voz e realizar tarefas como agendar compromissos, enviar mensagens e fornecer informações sobre o tempo e o tráfego. Ela foi projetada para ser uma assistente pessoal inteligente, integrada ao ecossistema Windows, e que utilizava o Aprendizado Profundo para aprimorar sua compreensão da linguagem humana ao longo do tempo, aprendendo a responder de maneira mais precisa às preferências do usuário.

1.3.14 Siri

A **Siri**, da Apple, também foi uma das primeiras assistentes virtuais a se tornar amplamente disponível, com seu lançamento em 2010. Combinando NLP com reconhecimento de fala, Siri possibilitou que os usuários interagissem com seus dispositivos iOS de maneira mais intuitiva, utilizando apenas comandos de voz. Ao longo dos anos, a Siri evoluiu, tornando-se mais precisa e capaz de realizar uma gama maior de tarefas, desde definir alarmes até responder a perguntas complexas.

1.3.15 Alexa

A **Alexa**, da Amazon, teve um impacto ainda mais amplo, popularizando o conceito de assistentes virtuais em dispositivos domésticos. Lançada em 2014 com o Amazon Echo (exemplo de um dispositivo de IoT), a Alexa utiliza o NLP para entender e executar comandos de voz, como tocar música, controlar dispositivos de automação residencial e até fazer compras online. Sua integração com uma vasta gama de serviços e dispositivos, além de sua habilidade de aprender com as interações dos usuários, fez dela uma das assistentes virtuais mais populares da década, ajudando a consolidar a voz como uma interface central para a interação com a tecnologia.

1.3.16 Google Assistant

Outro exemplo notável na evolução dos assistentes virtuais é o **Google Assistant**, lançado em 2016. Integrando o poder de busca do Google com avanços em Aprendizado Profundo e NLP, o Google Assistant foi projetado para oferecer respostas mais precisas, entender o contexto de conversas prolongadas e realizar ações em dispositivos móveis e domésticos inteligentes. Uma de suas maiores forças é sua integração com o ambiente Android (sistema operacional móvel), o que permite uma experiência fluida e personalizada para os usuários de smartphones e tablets. O assistente também se comunica de forma eficiente com outros dispositivos conectados, como smart speakers, smart TVs e sistemas de automação residencial, criando um ecossistema interconectado que se adapta às necessidades do usuário.

1.3.17 Microsoft Tay

Um exemplo de chatbot de Aprendizado Contínuo apresentado nessa década foi o **Microsoft Tay**. Ele foi um sistema de NLP com Inteligência Artificial lançado pela Microsoft no Twitter em 2016, projetado para interagir com usuários e aprender com as conversas, utilizando Aprendizado de Máquina, especificamente Redes Neurais. O sistema foi alimentado com dados de interações em tempo real, permitindo-lhe gerar respostas contextuais e adaptativas. No entanto, devido à ausência de filtros eficazes e moderadores para impedir entradas de usuários maliciosos, Tay aprendeu e reproduziu comportamentos tóxicos, como mensagens racistas e misóginas. Esses usuários exploraram a vulnerabilidade do modelo — que não conseguiu distinguir conteúdos ofensivos —, resultando em uma rápida escalada de mensagens prejudiciais. A Microsoft desativou o bot em menos de 24 horas, evidenciando falhas na abordagem de moderação em sistemas de IA baseados em Aprendizado Contínuo e aberto. Este foi um dos exemplos mais impactantes de que o desenvolvimento da tecnologia de NLP e IA não estava imune a questões éticas e sociais, nem ao comportamento dos usuários na rede.

1.3.18 Watson Assistant

Os sistemas NLP desta década, apesar de fortemente baseados em Aprendizado de Máquina, herdaram funcionalidades dos sistemas baseados em regras. Entretanto, uma das principais diferenças para os sistemas da década anterior é que esses softwares foram desenvolvidos no contexto de **computação distribuída**, ou seja, foram feitos para operarem nativamente em nuvem. Desta forma, foi possível que muitos dos sistemas discutidos nos parágrafos anteriores fossem transformados em plataformas de serviços para que qualquer empresa pudesse desenvolver sua própria versão de chatbots ou assistente virtual. Por exemplo, o **Watson Assistant**, da IBM, é uma plataforma robusta e baseada em Inteligência Artificial, que facilita a criação de chatbots e assistentes virtuais. Ele utiliza o Aprendizado de Máquina para melhorar a precisão das respostas ao longo do tempo, oferecendo uma interface visual para definir intenções e entidades que permitem o sistema entender o que o usuário está falando. O Watson também possui integração nativa com outros serviços da IBM, como o Watson Discovery, permitindo enriquecer as conversas com dados

extraídos de documentos. A plataforma oferece suporte a várias linguagens e é altamente escalável, adequada para empresas que precisam de uma solução empresarial avançada.

1.3.19 Google DialogFlow

O **Google DialogFlow** é uma plataforma do Google Cloud que se destaca pela simplicidade de uso e pela poderosa integração com os serviços do Google. Implementa funções de NLP para processar as intenções do usuário e retornar respostas relevantes. O DialogFlow oferece duas versões: uma básica e uma premium, com a versão premium incluindo mais recursos como integração com canais de comunicação, como o Facebook Messenger, e melhores opções de treinamento de modelos. A plataforma também é bem conhecida por causa da sua facilidade de integração com outras soluções de Inteligência Artificial do Google, como o Google Cloud Storage e o Google BigQuery, tornando-a uma boa escolha para projetos que requerem análise de dados em grande escala.

1.3.20 Rasa

Uma solução open-source é o **Rasa**, uma plataforma focada em flexibilidade e controle total sobre o sistema conversacional. Diferente do Watson Assistant e do DialogFlow, que são soluções SaaS (Software as a Service, ou em português “software como serviço”), o Rasa permite que os desenvolvedores hospedem o sistema em seus próprios servidores, o que é uma vantagem para empresas com necessidades específicas de privacidade ou segurança. Além disso, ele oferece ferramentas robustas para treinamento de modelos de NLP, permite o uso de scripts personalizados e a integração fácil com APIs externas.

1.3.21 BERT

No final da década surgiram os **modelos de atenção e transformers**, com destaque para o **BERT (Bidirectional Encoder Representations from Transformers)**, lançado pelo Google em 2018. O BERT revolucionou o NLP ao permitir que os modelos aprendessem representações profundas de contexto,

analisando simultaneamente o que vem antes e depois de cada palavra. Esse tipo de abordagem se mostrou eficaz em uma ampla gama de tarefas, desde perguntas e respostas até sumarização automática e detecção de sentimentos, e inaugurou a era dos **modelos pré-treinados**, que podiam ser ajustados (fine-tuned) para tarefas específicas com grande eficiência.

1.3.22 RoBERTa

Nos anos seguintes ao lançamento do BERT, surgiram várias outras propostas que procuraram melhorar ou expandir suas capacidades. Uma das primeiras inovações foi o **RoBERTa**, lançado em 2019 pelo Facebook AI Research. RoBERTa foi uma versão otimizada do BERT, com algumas modificações importantes no treinamento, como o uso de mais dados, mais tempo de treinamento e a remoção da tarefa de predição de Next Sentence Prediction (NSP), que o BERT utilizava. Esses ajustes permitiram que o RoBERTa superasse o BERT em diversos benchmarks, demonstrando um desempenho ainda melhor em tarefas de compreensão de texto, como perguntas, respostas e análise de sentimentos.

1.3.23 T5

Ainda em 2019, o **T5** (Text-to-Text Transfer Transformer), também desenvolvido pelo Google, propôs uma abordagem unificada para resolver todas as tarefas de NLP como problemas de “texto para texto”. Ou seja, todas as tarefas, desde tradução e sumarização até perguntas e respostas, eram tratadas de forma semelhante, em que o modelo recebe um texto de entrada e gera um texto de saída. Essa abordagem simplificou bastante o processo de treinamento e adaptação do modelo a diferentes tipos de tarefas, tornando-o uma solução poderosa e versátil para o NLP.

1.3.24 GPT-2 e GPT-3

Outro grande marco foi o lançamento do **GPT-2** pela OpenAI em 2019 e do **GPT-3** em 2020. Diferente do BERT, que é focado em entender o texto, os modelos da série **GPT** são voltados para a geração de texto, daí o nome Generative Pre-trained

Transformer (transform generativo pré-treinado). O GPT-2 já impressionava por sua capacidade de gerar textos coerentes e criativos em grande escala, enquanto o GPT-3, com 175 bilhões de parâmetros, levou isso a um novo nível, demonstrando uma versatilidade impressionante. Ele foi capaz de realizar uma variedade de tarefas de NLP, desde tradução até redação criativa, incluindo até mesmo resolução de problemas matemáticos simples, tudo isso com pouquíssimos exemplos de treinamento (o conceito de “*few-shot learning*”). O GPT-3, por sua vez, gerou grande entusiasmo não apenas pelo seu desempenho, mas também pela sua flexibilidade em ser aplicado em diversas tarefas sem a necessidade de ajustes finos específicos.

1.3.25 BART

Outro modelo relevante da mesma época foi o **BART** (Bidirectional and Auto-Regressive Transformers), também lançado pelo Facebook AI Research em 2019. O BART é uma arquitetura híbrida que combina características de modelos bidirecionais, como o BERT, e modelos autorregressivos, como o GPT. Isso permitiu que o BART fosse eficaz tanto em tarefas de compreensão quanto em tarefas de geração de texto. Sua aplicação se mostrou especialmente útil em tarefas como sumarização automática de textos, onde ele se destacou como um dos melhores modelos disponíveis. O BART, portanto, trouxe o melhor dos dois mundos: a capacidade de entender profundamente o texto (como o BERT) e de gerar texto de maneira coerente (como o GPT).

1.3.26 ALBERT

Em termos de eficiência, o **ALBERT** (A Lite BERT), também lançado em 2019 pelo Google, buscou reduzir a complexidade do BERT, mantendo a eficácia em tarefas de NLP. Para isso, o ALBERT compartilha pesos entre as camadas e reduz o número de parâmetros, tornando o modelo mais leve e rápido para treinar, sem perder muito da sua precisão. Esse modelo foi particularmente atraente para aplicações que precisavam de modelos mais rápidos e eficientes, sem comprometer a qualidade.

1.3.27 DistilBERT

Outro modelo de destaque foi o **DistilBERT**, desenvolvido pela Hugging Face em 2019. O DistilBERT se propôs a ser uma versão compacta do BERT, treinada por meio de uma técnica chamada “distilação do conhecimento”, onde um modelo maior é utilizado para treinar um modelo menor e mais eficiente. O resultado foi um modelo que, embora menor, manteve um desempenho muito próximo ao do BERT em várias tarefas, tornando-o ideal para aplicações com recursos computacionais limitados.

1.3.28 GPT-3.5 e MetaAI

Nos últimos cinco anos da década de 2020, a evolução dos modelos de Linguagem de Grande Escala (LLMs, do inglês Large Language Models) tem proporcionado uma revolução no campo do Processamento de Linguagem Natural, com o surgimento de modelos pré-treinados ainda mais poderosos e versáteis. Esses modelos, com sua capacidade de aprender e gerar textos com alto grau de coerência, têm sido o alicerce para uma série de aplicações práticas, desde assistentes virtuais até sistemas de recomendação e análise de sentimentos. Um exemplo notável dessa transformação é o **GPT-3.5**, desenvolvido pela OpenAI, que combina uma arquitetura avançada com uma enorme base de conhecimento, permitindo interações dinâmicas e realistas com os usuários.

Modelos como o ChatGPT e o **MetaAI** (da Meta, holding do Facebook e do WhatsApp) são alimentados por essas Redes Neurais de grande escala, que podem ser ajustadas para uma infinidade de tarefas, como responder perguntas complexas, redigir textos criativos, traduzir idiomas, e até mesmo fornecer assistência em tarefas técnicas ou acadêmicas. O avanço desses sistemas é impressionante, pois, mesmo com poucos exemplos de entrada, eles conseguem realizar uma quantidade impressionante de tarefas com precisão. Essa flexibilidade é especialmente valiosa para as empresas e organizações, que agora podem integrar essas soluções em seus produtos e serviços para melhorar a interação com os usuários, sem a necessidade de treinamentos específicos para cada nova tarefa.

1.3.29 Copilot

O **Copilot**, desenvolvido pela Microsoft em parceria com a OpenAI, e lançado em fevereiro de 2023, é uma aplicação prática e específica dos avanços de LLMs como o GPT, especialmente voltada para o desenvolvimento de software. A ideia de integrar um modelo de linguagem em um ambiente de programação surgiu da necessidade de melhorar a produtividade dos desenvolvedores, reduzindo o tempo gasto em tarefas repetitivas e fornecendo assistência inteligente na escrita de códigos. O Copilot, alimentado por um modelo de LLM pré-treinado, oferece sugestões contextuais de código em tempo real, tornando-se uma ferramenta poderosa para programadores, principalmente em plataformas como o GitHub, onde a colaboração e o gerenciamento de código são essenciais. A evolução dessas arquiteturas resultou em novas ideias de assistentes interativos: o Copilot, além de atuar como assistente de código, foi integrado ao Windows, substituindo a Cortana — descontinuada em 2024 —, e passou a operar com um app adicional ou como assistente dentro do navegador Microsoft Edge. Além de entender e gerar código e texto, suas capacidades foram expandidas para além do NLP de forma que o sistema também consiga entender e gerar imagens.

1.3.30 Gemini

O **Gemini**, desenvolvido pelo Google DeepMind e lançado em dezembro de 2023, representa uma das iniciativas mais avançadas da empresa no campo dos modelos de linguagem. Lançado como sucessor da linha de modelos **Bard**, o Gemini foi projetado para combinar um desempenho robusto com uma compreensão profunda da linguagem natural. Diferente de seus predecessores, o Gemini integra tecnologias de aprendizado multimodal, permitindo não apenas a análise e a geração de texto, mas também a compreensão de imagens e vídeos, o que amplia consideravelmente suas aplicações.

1.3.31 ChatGPT-4

O **ChatGPT-4**, lançado pela OpenAI em março de 2023, também é um exemplo de modelo de linguagem multimodal, ou seja, ele é capaz de processar tanto texto quanto imagens. Essa evolução em relação ao seu predecessor, o ChatGPT-3.5,

permite que o GPT-4 entenda e gere respostas não apenas baseadas em entradas textuais, mas também a partir de imagens, oferecendo uma experiência mais rica e versátil.

Esses modelos pré-treinados têm se tornado cada vez mais acessíveis, com empresas oferecendo APIs e plataformas que permitem a personalização e integração de LLMs em diversas aplicações, desde chatbots em sites de e-commerce até assistentes virtuais em dispositivos domésticos inteligentes. À medida que esses sistemas se tornam mais comuns, o limite de suas capacidades continua a se expandir, trazendo novas possibilidades para a automação de tarefas complexas e melhorando a forma como interagimos com a tecnologia no nosso dia a dia. A fusão entre IA e interface de usuário tem se destacado como um dos maiores avanços na experiência do usuário nas últimas décadas, gerando inclusive discussões geopolíticas.

1.3.32 DeepSeek

O **DeepSeek**, com sua abordagem inovadora e custo significativamente mais baixo para treinamento de modelos, tem se destacado como um exemplo de como empresas emergentes podem desafiar gigantes estabelecidos no campo da Inteligência Artificial. Seu modelo DeepSeek-R1, lançado em janeiro de 2025, trouxe uma nova perspectiva ao mercado, não apenas pela qualidade das respostas que gera, mas também pela maneira como superou limitações tecnológicas e financeiras, utilizando técnicas como mistura de especialistas (MoE) e chips de AI com desempenho inferior, devido às restrições comerciais impostas contra a China, país sede da DeepSeek. Isso provocou reações no setor, afetando diretamente empresas como a Nvidia, uma das maiores fornecedoras de chips para IA. Além disso, a estratégia de tornar os modelos “open weight”, compartilhando abertamente os parâmetros, atraiu a atenção de pesquisadores e desenvolvedores, colocando o DeepSeek na vanguarda da inovação em IA aberta. O impacto de DeepSeek no mercado global de IA tem sido um ponto central em debates geopolíticos, especialmente no que diz respeito às políticas de exportação de tecnologia e ao papel crescente da China no desenvolvimento de Inteligência Artificial de ponta.

Todos os sistemas mais avançados atuais herdam características das tecnologias predecessoras: eles implementam regras, Modelos de Aprendizado,

NLP e Chatbots: Vozes Digitais que Compreendem

Redes Neurais Profundas e computação distribuída. As principais empresas por trás dessas tecnologias disponibilizam plataformas em nuvem para que diferentes usuários possam aplicar as tecnologias em versões próprias de sistemas conversacionais ou outras aplicações de NLP. Algo que mudou ao longo da história do desenvolvimento dessas tecnologias é que, inicialmente, os avanços ocorriam nas universidades e nos centros de pesquisas, com entusiastas podendo gerar grandes contribuições nos computadores de seus próprios quartos. Entretanto, a partir de 2010, os avanços se restringiram às grandes empresas de tecnologia com equipes de pesquisa numerosas e milhões de dólares investidos.

A seguir, a figura ilustra uma linha do tempo contendo os principais desenvolvimento apresentados nessa seção.

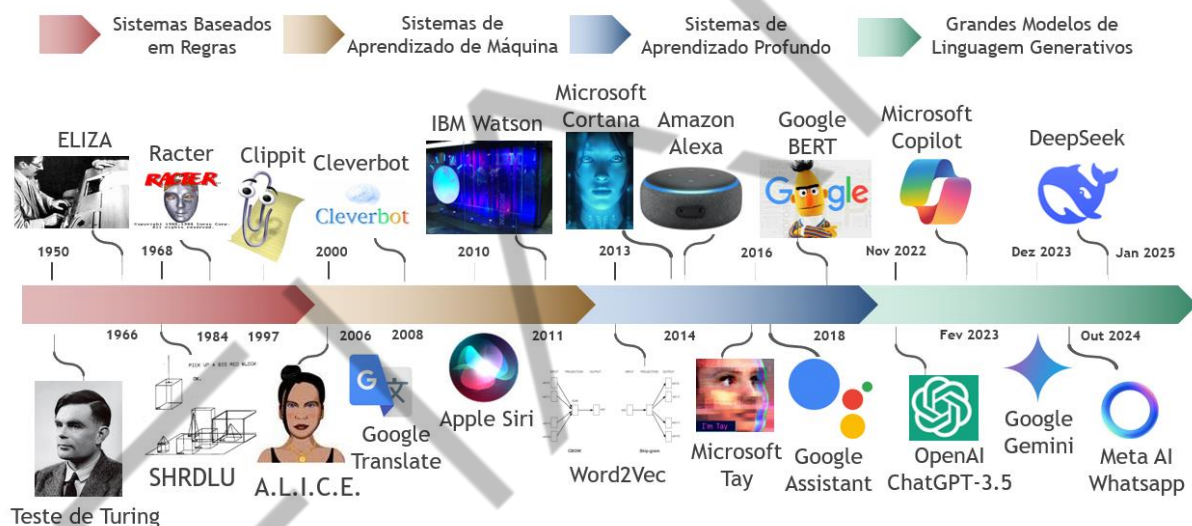


Figura 1 - Linha do tempo das tecnologias NLP
Fonte: Elaborado pelo autor (2025)

2 APLICAÇÕES DE NLP

2.1 Corretores Ortográficos

Corretores ortográficos (conhecidos em inglês como spell checkers), são ferramentas que detectam e corrigem erros de ortografia em textos escritos em linguagem natural. Eles são comuns em editores de texto, navegadores e aplicativos de mensagens.

Existem basicamente dois tipos principais de spell checkers. O primeiro é o **corretor ortográfico gramatical**, que funciona com base em regras ou em um dicionário de palavras válidas. Nesse caso, o sistema verifica se as palavras digitadas existem no dicionário. Se uma palavra não for reconhecida, ela é marcada como incorreta e o sistema sugere alternativas semelhantes. Essas sugestões são geradas com base em medidas como a distância de edição, que calcula quantas alterações são necessárias para transformar a palavra errada em uma palavra correta. Por exemplo, se alguém digita "teh", o sistema pode sugerir "the", pois há apenas uma pequena alteração a ser feita.

O segundo tipo é o **corretor ortográfico contextual**, mais moderno e sofisticado, que utiliza modelos de linguagem baseados em Aprendizado de Máquina. Esses modelos analisam não apenas a palavra isoladamente, mas também o contexto em que ela está inserida. Isso permite que o sistema identifique erros mais sutis, como quando a palavra está ortograficamente correta, porém, inadequada para o contexto. Um exemplo disso seria a frase "Eu vou concertar o carro amanhã", em que "concertar" está corretamente escrita, mas o contexto indica que a palavra correta seria "consertar". Modelos desse tipo geralmente utilizam técnicas como embeddings de palavras (por exemplo, Word2Vec ou BERT) com Redes Neurais Artificiais para prever a palavra mais apropriada com base no sentido da frase.

2.2 Tradutores Automáticos

A tradução automática de texto é uma das aplicações mais populares e avançadas de NLP. O principal objetivo da tradução automática é converter textos de um idioma para outro de forma automática, preservando o significado original, o

contexto e o tom. Essa tarefa, embora pareça simples à primeira vista, envolve uma complexa interação de Modelos Linguísticos, Análise Semântica, Morfossintática e Aprendizado de Máquina.

Historicamente, os primeiros sistemas de tradução automática eram baseados em regras linguísticas rígidas — os sistemas baseados em regras, como já mencionado anteriormente. Esses modelos exigiam grande conhecimento gramatical dos idiomas envolvidos e um extenso trabalho manual para construir dicionários bilíngues e regras gramaticais. Posteriormente, surgiram os sistemas Statistical Machine Translation (SMT), como o famoso modelo Moses, que utilizam grandes corpora de textos paralelos (traduções humanas de textos entre dois idiomas) para calcular probabilidades estatísticas de correspondência entre palavras em frases.

Atualmente, os sistemas mais eficientes e amplamente utilizados se baseiam em técnicas de Aprendizado Profundo, especialmente os chamados modelos de Transformers. Esses modelos, como o BERT, o GPT e o T5, revolucionaram o campo da tradução automática com o uso de Redes Neurais Profundas e mecanismos de atenção, que permitem ao modelo considerar o contexto completo de uma frase ou parágrafo ao traduzir. A arquitetura encoder-decoder é fundamental nesse processo: o encoder codifica o texto de entrada em um vetor de representação densa, enquanto o decoder gera o texto traduzido palavra por palavra, baseando-se nesse vetor e no contexto já gerado.

Apesar dos avanços, ainda existem desafios significativos. Ambiguidade semântica, diferenças culturais, expressões idiomáticas e variações regionais dos idiomas são obstáculos frequentes. Além disso, a tradução de textos especializados, como documentos jurídicos ou científicos, exige modelos treinados em domínios específicos, pois a terminologia técnica deve ser mantida com precisão.

Podemos ver um exemplo dessas limitações na figura “Exemplo de compreensão diferente de expressão idiomática (diferença cultural) com o Microsoft Copilot gerando imagens por prompt de texto”, onde é pedido para um sistema generativo de texto e imagens uma tarefa em inglês e a mesma tarefa em português: como o modelo em inglês tem mais dados de treinamento do que sua versão em português, os desempenhos de compreensão apresentados são distintos.

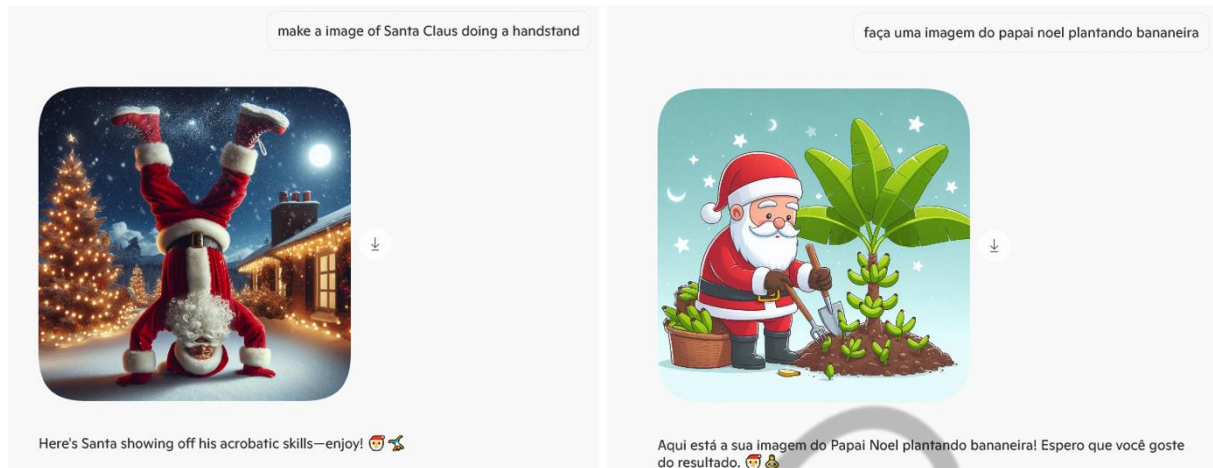


Figura 2 - Exemplo de compreensão diferente de expressão idiomática (diferença cultural) com o Microsoft Copilot gerando imagens por prompt de texto
Fonte: Elaborado pelo autor (2025)

2.3 Recuperação de Informação

A forma como buscamos informações tem evoluído drasticamente nos últimos anos. Tradicionalmente, os sistemas de busca eram baseados em palavras-chave exatas, exigindo que o usuário soubesse exatamente o que procurar e como formular sua consulta. Hoje, com o avanço do Processamento de Linguagem Natural, os sistemas de recuperação de informação (em inglês, **Information Retrieval**, IR) baseados em linguagem natural oferecem uma maneira muito mais intuitiva de buscar conteúdo: entendendo o significado por trás da pergunta, e não apenas as palavras que a compõem.

Sistemas modernos, como os **mecanismos de busca** do Google, assistentes virtuais (como Siri ou Alexa) e chatbots inteligentes, são capazes de interpretar consultas feitas em linguagem natural, como por exemplo “quais os impactos da mudança climática na agricultura brasileira?”. Esse tipo de pergunta contém múltiplas camadas semânticas, e o sistema precisa entender as intenções, relações entre conceitos e contexto para fornecer uma resposta relevante. Para isso, são utilizados modelos de Aprendizado de Máquina, como embeddings, que convertem texto em vetores de significado, permitindo que perguntas e respostas sejam comparadas com base em similaridade semântica.

Essa abordagem contrasta com a lógica de sistemas de bancos de dados tradicionais, como SQL e MongoDB. Nesses, a busca depende de estruturas bem

definidas e de comandos exatos. Por exemplo, para buscar todos os clientes com mais de 30 anos em SQL, você escreveria algo como:

```
SELECT * FROM clientes WHERE idade > 30
```

Esse tipo de busca é rápido e poderoso dentro de contextos estruturados, mas não lida bem com a flexibilidade da linguagem natural. Você não pode simplesmente perguntar “quais clientes são mais velhos?” — o sistema não entenderia a intenção. Além disso, bancos como MongoDB, mesmo sendo NoSQL e mais flexíveis em termos de estrutura de dados, ainda exigem consultas estruturadas e específicas, como:

```
db.clientes.find({idade: {$gt: 30}})
```

Visto que eles não interpretam o significado por trás das palavras ou expressões.

Por essa razão, os sistemas de **busca semântica** baseados em NLP não substituem os bancos de dados tradicionais, mas sim os complementam em contextos onde a interpretação de linguagem natural é importante — como buscadores, assistentes virtuais, suporte ao cliente e análise de grandes volumes de texto não estruturado.

No próximo quadro, apresentamos uma comparação entre as formas de busca que podem ser realizadas no contexto de Recuperação de Informação. Uma forma bastante tradicional de busca é aquela realizada por palavra-chave, que é quando apertamos Ctrl+F em um documento PDF e escrevemos a palavra que estamos procurando. Buscadores de internet também usam algo similar, mas procuram palavras-chaves em textos inseridos dentro de códigos HTML. Por esse motivo, é necessário estratégias de ranqueamento, visto que a quantidade de páginas web é infinitamente maior do que o número de páginas em um PDF (você já tentou buscar uma palavra em um documento com 50 páginas e a mesma palavra em um documento com 1000 páginas? — há uma grande diferença de tempo de execução do comando!). Com isso, torna-se evidente que a busca por palavras-chave é bastante limitada, apesar de útil.

Característica	Busca Tradicional	Busca por Linguagem Natural	Busca em Banco de Dados
Tipo de consulta	Palavras-chave exatas	Frases e perguntas em linguagem natural	Consultados estruturadas e precisas
Interpretação de contexto	Limitada	Alta — entende intenção e significado	Nenhuma — requer a estrutura exata
Flexibilidade na formulação da pergunta	Baixa	Alta	Baixa
Uso de sinônimos ou linguagem variada	Limitado	Suportado	Não suportado
Suporte a dados estruturados	Parcial	Raro (mas possível com integração)	Total
Exemplos	“clima” + “agricultura”	“Como o clima afeta a agricultura?”	<code>SELECT * FROM dados WHERE ...</code>
Casos de uso	Web Search simples	Atendimento, FAQ, documentos longos	Sistemas de registro, softwares de contabilidade

Quadro 1 - Comparação entre sistemas de Recuperação de Informação
Fonte: Elaborado pelo autor (2025)

2.4 Sumarização de Texto

Os sumarizadores de texto são sistemas ou algoritmos desenvolvidos para gerar automaticamente versões resumidas de textos mais longos. O principal objetivo desses sistemas é identificar e apresentar as informações mais relevantes do conteúdo original de forma mais curta e concisa, sem perder o sentido essencial do que foi dito. Eles constituem um exemplo da grande categoria de algoritmos de NLP denominados de **Extratores de Informação** — conhecidos em inglês como Information Extraction (IE).

Existem dois tipos principais de sumarização: a **extrativa** e a **abstrativa**. A sumarização extrativa funciona selecionando frases ou trechos exatos do texto original que são considerados mais importantes. Essa abordagem geralmente utiliza técnicas como frequência de palavras, posição das frases e algoritmos como TextRank ou TF-IDF. Sua principal vantagem é preservar exatamente a linguagem do texto original, porém, o resumo pode acabar parecendo fragmentado ou com pouca fluidez.

Já a sumarização abstrativa busca reformular o conteúdo original, criando novas frases com linguagem própria, como se estivesse recontando o texto de maneira resumida. Essa técnica é mais complexa e se baseia em modelos de

Aprendizado Profundo, como os Transformers (exemplos: BART, T5, GPT). Embora exija mais recursos computacionais, ela tende a gerar resumos mais naturais, coesos e parecidos com os que um ser humano faria.

Essas técnicas são amplamente utilizadas em diversos contextos, como na geração automática de resumos de notícias, no atendimento ao cliente para sintetizar chamados, em assistentes pessoais para resumir e-mails ou artigos, e até mesmo em aplicações acadêmicas.

2.5 Análise de Sentimento

A análise de sentimento é um exemplo de aplicação de algoritmos de NLP da categoria ER, mas, que neste caso, são utilizados para extrair informações subjetivas como emoções e sentimentos. A aplicação de NLP nesta tarefa tem se consolidado como uma das áreas mais relevantes dentro da Inteligência Artificial, especialmente para empresas que buscam compreender as opiniões de consumidores e a percepção pública sobre marcas, produtos e serviços. A análise de sentimento utiliza técnicas de NLP para classificar automaticamente os textos — como resenhas de produtos, posts em redes sociais ou interações em atendimento ao cliente — em **categorias de sentimentos, como positivo, negativo ou neutro**. Esse tipo de análise proporciona insights valiosos sobre a satisfação dos clientes, a reputação da marca e as tendências emergentes no mercado.

O processo de análise de sentimento começa com o pré-processamento de dados textuais, que envolve a remoção de ruído, como palavras irrelevantes (stopwords), símbolos ou pontuações que não contribuem diretamente para a análise. Em seguida, a tokenização divide o texto em unidades menores, como palavras ou frases, para facilitar a análise subsequente. Dependendo da complexidade do modelo, a normalização das palavras, que pode incluir a lematização (redução das palavras à sua forma base) ou stemming (corte das palavras para suas raízes), também é realizada para melhorar a precisão do modelo.

Após o pré-processamento, técnicas de Aprendizado de Máquina entram em ação. Modelos supervisionados, como Support Vector Machines (SVM) ou Naive Bayes, são treinados com conjuntos de dados rotulados (contendo exemplos de textos classificados como positivos, negativos ou neutros), enquanto técnicas mais

avançadas, como Redes Neurais Recorrentes (RNNs) e modelos baseados em transformadores, como o BERT, são capazes de capturar nuances mais sutis nos textos. Esses modelos são treinados para identificar padrões semânticos que indicam a polaridade do sentimento expresso no texto.

Além de classificar o sentimento de forma binária ou multinível, modelos de NLP podem ser configurados para detectar sentimentos mais complexos, como emoções específicas (raiva, tristeza, alegria) ou até mesmo ironia, que pode ser um grande desafio para os sistemas de Inteligência Artificial devido ao contexto ambíguo presente nas expressões humanas. O uso de embeddings de palavras, como o Word2Vec ou o GloVe, também ajuda a melhorar a captura de semelhanças semânticas entre termos, permitindo que o modelo entenda melhor os contextos em que uma palavra ou expressão específica pode mudar de sentido.

Uma das vantagens de utilizar NLP para análise de sentimento é a capacidade de realizar análises em tempo real, algo essencial em cenários de monitoramento de redes sociais ou de feedback contínuo de consumidores. As empresas podem usar essas informações para ajustar rapidamente suas estratégias de marketing, melhorar a experiência do cliente ou prever possíveis crises de imagem antes que elas se amplifiquem. Entretanto, os desafios ainda persistem. A diversidade de linguagem, a subjetividade e a ambiguidade das palavras tornam a análise de sentimento uma tarefa complexa. Por exemplo, o uso de palavras com múltiplos significados, gírias regionais, variações de tom, ou a presença de sarcasmo, são aspectos que podem confundir os algoritmos. Para solucionar essas questões, é necessário o uso de corpora especializados, como os dados de redes sociais ou interações de clientes, e abordagens que incorporem contexto mais amplo.

2.6 Resposta a Perguntas, Chatbots, Assistentes Virtuais e Agentes

Os **Sistemas de Resposta a Perguntas** (do inglês Question Answering Systems, QA) são uma área de crescente importância na automação da interação com usuários, oferecendo respostas precisas e contextualizadas a perguntas formuladas em linguagem natural. Esses sistemas envolvem o desenvolvimento de tecnologias que permitem a obtenção de respostas de forma automática e eficiente, sem a necessidade de intervenção humana direta. Essa abordagem está no centro de

muitas aplicações modernas, como chatbots, assistentes virtuais e agentes inteligentes, que buscam compreender e responder de maneira relevante e adequada aos usuários.

Os **chatbots** são sistemas automatizados projetados para interagir com humanos por meio de mensagens de texto ou voz. Tecnicamente, um chatbot tradicional é construído com base em regras pré-definidas e scripts, frequentemente utilizando técnicas de NLP como tokenização, análise sintática, Reconhecimento de Entidade Nomeada (NER) e classificação de intenções. Por exemplo, ao receber a entrada "Qual é o horário de funcionamento?", o chatbot identifica a intenção ("consultar horário") e extrai a entidade relevante (horário ou local), direcionando a resposta a partir de uma base de dados ou regras.

Os chatbots tradicionais normalmente operam dentro de domínios limitados, como atendimento ao cliente, reservas ou FAQs. Eles são rápidos, eficientes e relativamente baratos de implementar, mas carecem de flexibilidade e entendimento contextual profundo. Ainda hoje, muitos sistemas comerciais empregam esse tipo de chatbot combinado com NLP simbólico e modelos estatísticos.

Embora muitas vezes usados como sinônimos, os **assistentes virtuais** diferem dos chatbots em sua complexidade e capacidade de integração. Assistentes como a Siri, a Alexa e o Google Assistant vão além da simples conversação: eles interagem com o ecossistema do dispositivo, compreendem comandos multimodais, acessam e organizam dados do usuário, e podem realizar tarefas como configurar alarmes, tocar músicas ou controlar dispositivos domésticos inteligentes. Do ponto de vista técnico, esses sistemas integram NLP com **reconhecimento de fala, síntese de voz, gerenciamento de contexto multi-turno e sistemas de diálogo baseados em intenções**, utilizando algoritmos de Aprendizado de Máquina treinados com grandes volumes de dados de interações humanas.

Com o advento dos Large Language Models (LLMs) como GPT, Claude e Gemini, surgem os **Agentes LLM**, que representam um salto qualitativo em relação aos chatbots e assistentes tradicionais. Esses agentes não apenas compreendem e geram texto de forma fluente e contextualizada, mas também demonstram capacidades emergentes como raciocínio, planejamento, criação de código, resumos, traduções e muito mais. Em termos técnicos, os LLMs são modelos baseados em Transformers, treinados com bilhões ou trilhões de parâmetros, utilizando datasets

massivos e heterogêneos. Eles não operam com regras fixas, mas com representações vetoriais altamente distribuídas, o que lhes permite inferir padrões complexos de linguagem e até realizar tarefas para as quais não foram explicitamente treinados.

Além disso, os agentes LLMs muitas vezes são acoplados a ferramentas externas (por exemplo: plugins, APIs, navegadores, bancos de dados) e podem autonomamente decompor tarefas, decidir qual ferramenta usar e como interagir com ela — algo que transcende o paradigma dos chatbots.



3 CATEGORIZAÇÃO DOS ALGORITMOS DE NLP

3.1 Sistemas de Transformação

Nesta abordagem, podemos classificar os algoritmos de NLP **conforme o tipo de entrada de saída** que eles recebem e geram, respectivamente. Denominamos a abordagem de sistema pois imaginamos que o algoritmo é uma caixa-preta, da qual abstraímos os componentes internos, observando apenas as entradas (inputs) e saídas (outputs). Existem três tipos de sistemas:

- **Transformação texto para texto:** neste tipo de sistema, a entrada e a saída são não estruturadas, sendo ambas no formato de texto puro. Exemplo de sistemas reais que operam dessa forma são: tradutores automáticos, corretores ortográficos e chatbots.
- **Transformação de dados estruturados para texto:** neste tipo de sistema, a entrada é estruturada ou semi-estruturada. Isso significa que o sistema recebe uma variável bem comportada, como um vetor (lista ou array) ou um dicionário. A saída desse sistema é gerar texto puro a partir dessa entrada estruturada. Por exemplo, sistemas de geração automática de relatórios, onde a entrada é composta de dados numéricos ou tabelas, e a saída é um texto fluido que descreve os dados de maneira compreensível para um usuário. Outros exemplos incluem sistemas de geração de descrições de produtos a partir de atributos estruturados — como nome, preço, características, entre outros — ou assistentes virtuais que convertem informações de bancos de dados em respostas narrativas.
- **Transformação de texto para dados estruturados:** neste tipo de sistema, a entrada é um texto puro e a saída são dados estruturados. Neste caso, o sistema opera, por exemplo, em cima de um documento redigido e procura campos específicos para preencher um banco de dados relacional. Um exemplo são as Aplicações de Classificação de Texto, como a Análise de Sentimento.

3.2 Entendimento e Geração

Nesta abordagem, podemos classificar os algoritmos de NLP **conforme o objetivo principal da tarefa** realizada. Os Sistemas de Entendimento de Linguagem Natural — conhecidos em inglês pela sigla **NLU** (Natural Language Understanding) — permitem que o computador “entenda” as informações contidas em um texto puro. Já os Sistemas de Geração de Linguagem Natural — conhecidos em inglês pela sigla **NLG** (Natural Language Generation) — “escrevem” textos de forma a realizar a tarefa solicitada pelo usuário.

Em comparação com a categorização em Sistemas de Transformação, os sistemas NLU são aqueles que convertem texto para dados estruturados, enquanto os sistemas NLG são aqueles que convertem dados estruturados para texto. Entretanto, pode surgir uma pergunta: e os sistemas de transformação Texto para Texto? Na verdade, esses sistemas podem ser feitos de duas maneiras: **juntando-se um NLU seguido de um NLG** ou **utilizando apenas um NLG retroalimentado**.

Observe que, da perspectiva do usuário, um sistema puramente NLU pode parecer capaz de gerar textos. Um exemplo típico são chatbots com respostas pré-programadas. Algoritmos de NLP são usados para realizar a etapa de NLU de modo que utilize as informações extraídas sobre uma lógica programação tradicional, como Árvore de Decisão IF/ELSE. Dependendo da sequência de condicionais realizada, frases pré-programadas são apresentadas para o usuário.

Por outro lado, da perspectiva do usuário, sistemas puramente NLG podem ser interpretados como se tivessem a capacidade de compreender textos. Um exemplo típico são chatbots generativos por retroalimentação. Este tipo de sistema opera através da predição de uma palavra baseada nas N palavras anteriores, onde N é um número arbitrário (1, 2, 3 etc.). A retroalimentação envolve fazer o sistema receber a própria palavra que ele gerou, além das palavras que o usuário inseriu. Esse ciclo se repete até que uma palavra especial que simboliza o “fim de operação” seja gerada pelo módulo preditor. Assim, o sistema devolve para o usuário somente a parte contendo as palavras que ele gerou durante os ciclos de realimentação. Para o usuário parece que o sistema de fato entendeu o que ele disse, quando, na verdade, o sistema apenas gerou um texto que possui uma alta probabilidade de aparecer exatamente na frente das palavras que o usuário inseriu no sistema.

Na figura a seguir, apresentamos um diagrama esquemático com os três tipos de sistemas de transformação em NLP categorizados também de acordo com a capacidade de entendimento e de geração de texto.

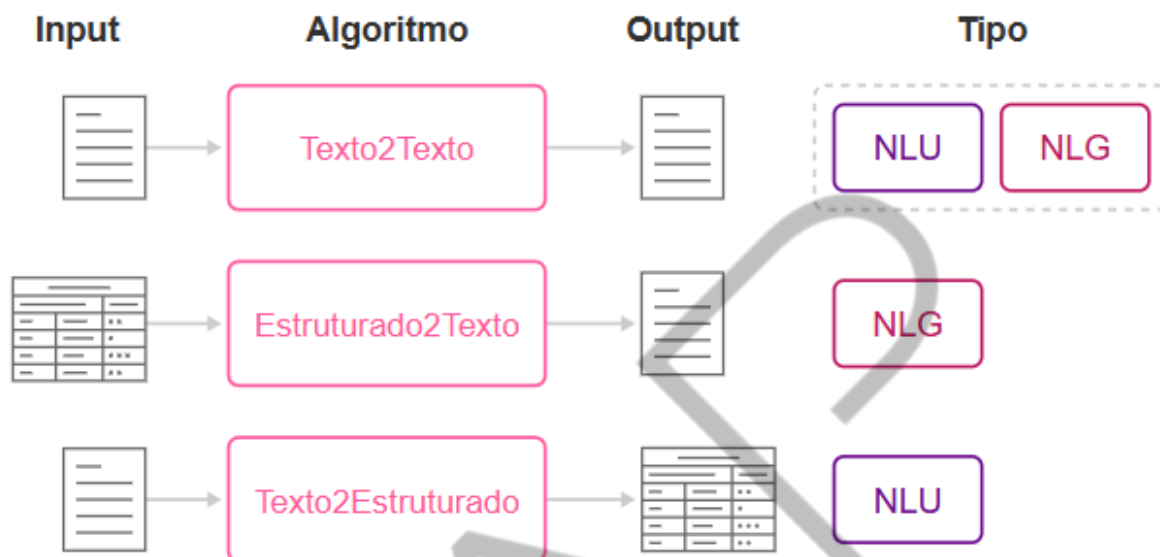


Figura 3 - Esquemático para categorizar algoritmos de NLP de acordo com o tipo de entrada e saída e a capacidade de entendimento e geração de texto

Fonte: Elaborado pelo autor (2025)

3.3 Recuperação e Extração de Informação

A principal diretriz de um algoritmo NLP é a capacidade de processar linguagem natural. Alguns algoritmos de NLP são usados para Recuperar Informação (IR), como foi discutido no tópico “Aplicações de NLP” deste capítulo. Entretanto, precisamos avisar que nem todos os sistemas de IR são algoritmos de NLP. Na verdade, a área de IR possui uma dinâmica própria que vai além da linguagem natural, envolvendo categorização de conteúdo, bancos de dados relacionais e não relacionais, hardware para armazenamento de dados, velocidade de recuperação de informação, entre outras tarefas. Além disso, ao recuperar informações, buscamos retorná-las sem modificações! Ou seja, queremos os dados no formato original em que estão armazenados.

Já a Extração de Informação (IE) é uma categoria de algoritmo utilizado para transformar um dado não estruturado em uma informação estruturada ou semi-estruturada. Na Visão Computacional, por exemplo, se estivermos interessados em identificar se uma foto possui um gato ou um cachorro, o algoritmo deve entender a imagem (um dado não estruturado) e gerar duas saídas possíveis e bem comportadas

(estruturadas): gato ou cachorro. Isso é chamado de Classificação de Imagens. Em NLP podemos realizar o mesmo tipo de tarefa, naquilo que é chamado de Classificação de Texto. A análise de sentimentos também constitui um exemplo de algoritmo de Extração de Informação em NLP por transformar dado não estruturado (texto) em um dado estruturado (classes de sentimento ou classes de emoção).

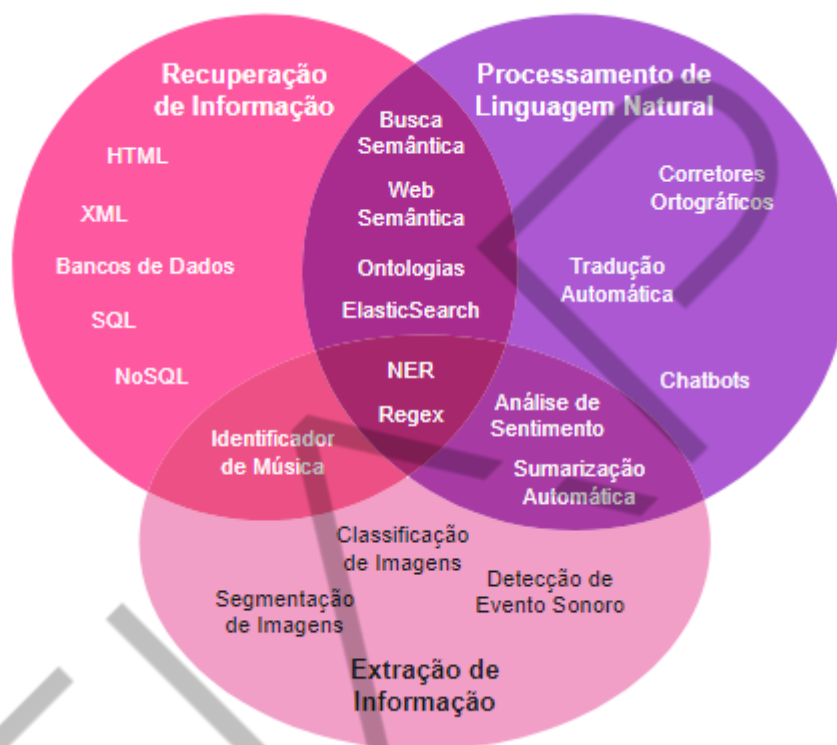


Figura 4 - Diagrama de Venn das categorias de algoritmos de NLP de acordo com a capacidade de recuperar ou extrair informação
Fonte: Elaborado pelo autor (2025)

Na figura “Diagrama de Venn das categorias de algoritmos de NLP de acordo com a capacidade de recuperar ou extrair informação” é esquematizada a discussão sobre as categorias de algoritmos de NLP quanto a extração e recuperação de informação.

Importante: todas as categorizações discutidas nessa seção devem ser consideradas com senso crítico. Por exemplo, um chatbot pode extrair ou recuperar informação?

4 GRAMÁTICAS FORMAIS E EXPRESSÕES REGULARES

4.1 Hierarquia de Chomsky

As hierarquias de Chomsky são uma classificação das gramáticas formais com base na complexidade de suas produções e nos tipos de linguagens que elas conseguem gerar. Essa classificação foi proposta pelo linguista Noam Chomsky em 1956, com o objetivo de categorizar as gramáticas e ajudar a entender os diferentes níveis de poder de expressão das linguagens formais. A hierarquia é composta por quatro níveis, que vão desde gramáticas mais simples e limitadas até gramáticas mais poderosas, que podem gerar linguagens mais complexas.

- **Gramáticas Tipo 0: Gramáticas Irrestritas**

- **Definição:** são as gramáticas mais gerais e não possuem restrições sobre as formas das produções. Podem ter qualquer tipo de regra de produção, desde que a regra tenha a forma $\alpha \rightarrow \beta$, onde α e β são cadeias de símbolos (com α podendo ser uma cadeia não vazia).
- **Linguagens:** as linguagens geradas por gramáticas tipo 0 são chamadas de linguagens recursivamente enumeráveis. Elas incluem todas as linguagens que podem ser geradas por qualquer tipo de máquina de Turing, mas não são necessariamente decidíveis.
- **Exemplo:** qualquer linguagem que possa ser descrita por uma máquina de Turing.

- **Gramáticas Tipo 1: Gramáticas Sensíveis ao Contexto**

- **Definição:** as regras de produção dessas gramáticas têm a forma $\alpha A \beta \rightarrow \alpha \gamma \beta$, onde A é um símbolo não terminal, α , β , e γ são cadeias de símbolos, e γ não pode ser a cadeia vazia. Em outras palavras, o lado esquerdo da produção pode ter mais de um símbolo, mas o lado direito precisa ter pelo menos a mesma quantidade de símbolos do lado esquerdo (ou mais).
- **Linguagens:** as linguagens geradas por gramáticas tipo 1 são chamadas de linguagens sensíveis ao contexto. Elas são mais poderosas do que as

linguagens regulares e as linguagens livres de contexto, mas menos poderosas que as linguagens recursivamente enumeráveis.

- **Exemplo:** linguagens que podem ser reconhecidas por uma máquina de Turing não determinística, onde o contexto das produções afeta as transições.

- **Gramáticas Tipo 2: Gramáticas Livres de Contexto**

- **Definição:** as produções de uma gramática livre de contexto têm a forma $A \rightarrow \gamma$, onde A é um símbolo não terminal e γ é uma cadeia de símbolos (que pode ser composta por terminais e não terminais). Ou seja, a produção depende apenas do símbolo não terminal à esquerda, independentemente do contexto em que ele aparece.
- **Linguagens:** as linguagens geradas por gramáticas tipo 2 são chamadas de **linguagens livres de contexto**. Essas linguagens são muito importantes para a teoria da computação e para a descrição de muitas linguagens de programação e linguagens naturais. Elas podem ser reconhecidas por autômatos de pilha, como os utilizados em compiladores.
- **Exemplo:** a linguagem de parênteses balanceados, como $\{(), (()), (((()))\}$, é uma linguagem livre de contexto; análise sintática realizada por compiladores.

- **Gramáticas Tipo 3: Gramáticas Regulares**

- **Definição:** as produções dessas gramáticas são da forma $A \rightarrow aB$ ou $A \rightarrow a$, onde A e B são símbolos não terminais e ' a ' é um símbolo terminal. Ou seja, as produções são muito restritas e os não terminais só podem aparecer no final ou no início de uma cadeia.
- **Linguagens:** as linguagens geradas por gramáticas tipo 3 são chamadas de linguagens regulares. Elas podem ser reconhecidas por autômatos finitos, que são os tipos de máquinas mais simples e rápidos para processamento de linguagem. São usadas, por exemplo, em expressões regulares, que são amplamente usadas para busca e substituição de padrões em texto.

- **Exemplo:** a linguagem dos números binários que representam números divisíveis por 3 é regular; análise léxica realizada por compiladores.

A figura a seguir apresenta um diagrama de Venn desta hierarquia.

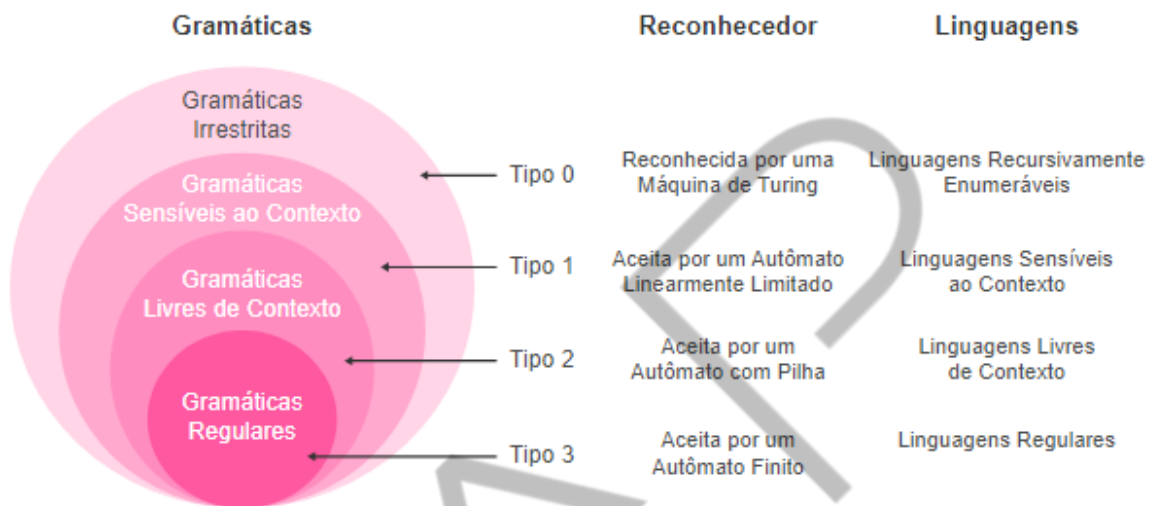


Figura 5 - Diagrama de Venn com a categorização de Chomsky das gramáticas formais
Fonte: Elaborado pelo autor (2025)

A relação hierárquica entre as gramáticas de Chomsky segue uma estrutura de inclusão, ou seja, quanto mais restritiva for a gramática, menos poder ela tem para gerar linguagens, mas mais fácil é de ser processada.

As hierarquias de Chomsky são fundamentais para a teoria da computação e a linguística computacional. Elas ajudam a entender os limites das máquinas e algoritmos, além de fornecer uma base para a análise e desenvolvimento de linguagens de programação, compiladores e outras ferramentas de processamento de linguagem.

Compiladores e Paradigmas de Programação são as áreas que se concentram principalmente em linguagens regulares e livres de contexto, pois essas são suficientes para representar a sintaxe de linguagens de programação. Já o Processamento de Linguagem Natural lida com linguagens que, devido à sua complexidade e nuances, exigem técnicas que podem envolver linguagens sensíveis ao contexto ou irrestritas. Devido às dificuldades computacionais, as abordagens de NLP frequentemente utilizam Técnicas Probabilísticas, Redes Neurais e outros métodos para lidar com a flexibilidade e ambiguidade da linguagem humana, podendo

ir além das regras rígidas estabelecidas dentro de cada classificação de gramática formal.

Importante: no NLP, as abordagens computacionais práticas são muito mais complexas e não se limitam a uma simples aplicação da hierarquia de Chomsky.

4.2 Expressões regulares

Expressões regulares, conhecidas em inglês pelo acrônimo **regex**, são padrões de pesquisa utilizados para buscar, corresponder, manipular e substituir texto em cadeias de caracteres. Foram inventadas por Stephen Cole Kleene, um matemático e lógico americano, na década de 1950, como parte de seu trabalho no campo da teoria da computação. Ele as introduziu no contexto da teoria de autômatos e como uma maneira de descrever padrões em sequências de símbolos. As expressões regulares seguem uma sintaxe formal que define como os caracteres podem ser combinados de maneiras específicas. Em termos simples, uma expressão regular é uma sequência de caracteres que forma um padrão de busca. As expressões regulares podem descrever linguagens regulares, que são o Tipo 3 da hierarquia de Chomsky. Linguagens regulares são aquelas que podem ser reconhecidas por um autômato finito determinístico. Esses autômatos não têm memória além do estado atual, isso significa que eles só conseguem processar uma sequência de caracteres de forma linear e sem dependência de contexto.

Após o trabalho de Kleene, as expressões regulares começaram a ser utilizadas em várias áreas da computação, especialmente na **análise léxica** e na **análise sintática** de linguagens de programação. Elas se tornaram uma ferramenta indispensável para programadores e desenvolvedores devido à sua capacidade de realizar **buscas e substituições** eficientes em strings de texto. O conceito de expressões regulares ganhou popularidade no ambiente Unix, onde foi incorporado em ferramentas como `grep`, `sed`, e `awk`. Essas ferramentas permitiram aos desenvolvedores usar expressões regulares para buscar, manipular e transformar texto de maneira poderosa e eficiente. O regex tornou-se uma parte fundamental de muitas linguagens de programação, como Perl, Python, JavaScript, Ruby e outras, tornando-se uma ferramenta padrão para a manipulação de strings.

A implementação de expressões regulares em Python é fornecida pelo módulo `re`, que oferece uma sintaxe muito próxima das expressões regulares padrão, com alguns detalhes específicos. A biblioteca `re` já é um módulo nativo do Python, logo, não é necessária nenhuma instalação. Agora vamos abordar os **metacaracteres** e suas funções na implementação de regex do Python. No código-fonte a seguir, demonstramos o uso das principais funções da biblioteca `re` e os metacaracteres usados.

```
import re

# re.match(pattern, string): Verifica se a expressão
regular combina com o início da string.
# re.search(pattern, string): Procura a expressão regular
em qualquer lugar da string.
# re.findall(pattern, string): Retorna uma lista de todas
as correspondências não sobrepostas da expressão regular na
string.
# re.sub(pattern, replacement, string): Substitui todas
as ocorrências de um padrão na string por um texto de
substituição.

# -----
# Metacarater . corresponde a qualquer caractere (exceto
uma nova linha)
pattern = r"a.c"
result = re.match(pattern, "abc") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "ac") # Não corresponde
print(result) # Saída: None

# -----
# Metacarater ^ corresponde ao início da string
pattern = r"^abc"
result = re.match(pattern, "abcdef") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "zabcdef") # Não corresponde
print(result) # Saída: None

# -----
# Metacarater $ corresponde ao final da string
pattern = r"abc$"
result = re.match(pattern, "abcdef") # Não corresponde
print(result) # Saída: None
result = re.match(pattern, "abcxyz") # Não corresponde
print(result) # Saída: None
result = re.search(pattern, "abcxyz") # Corresponde
```

NLP e Chatbots: Vozes Digitais que Compreendem

```
print(result)    # Saída: <re.Match object; span=(3, 6),
match='abc'>

# -----
# Metacarater * corresponde a zero ou mais ocorrências do
caractere ou grupo anterior
pattern = r"ab*c"
result = re.match(pattern, "abc") # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "ac")  # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 2),
match='ac'>
result = re.match(pattern, "abbc") # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 4),
match='abbc'>

# -----
# Metacarater + corresponde a uma ou mais ocorrências do
caractere ou grupo anterior
pattern = r"ab+c"
result = re.match(pattern, "abc") # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "ac")  # Não corresponde
print(result)    # Saída: None
result = re.match(pattern, "abbc") # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 4),
match='abbc'>

# -----
# Metacarater ? corresponde a zero ou uma ocorrência do
caractere ou grupo anterior
pattern = r"ab?c"
result = re.match(pattern, "abc") # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "ac")  # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 2),
match='ac'>
result = re.match(pattern, "abbc") # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater [] define um conjunto de caracteres
pattern = r"[abc]"
result = re.match(pattern, "a")  # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 1),
match='a'>
result = re.match(pattern, "b")  # Corresponde
```

NLP e Chatbots: Vozes Digitais que Compreendem

```
print(result)    # Saída: <re.Match object; span=(0, 1),
match='b'>
result = re.match(pattern, "z")    # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater | representa uma alternativa (OU)
pattern = r"abc|def"
result = re.match(pattern, "abc")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 3),
match='abc'>
result = re.match(pattern, "def")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 3),
match='def'>
result = re.match(pattern, "xyz")    # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater () para agrupar padrões
pattern = r"(ab)+"
result = re.match(pattern, "ababab")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 6),
match='ababab'>
result = re.match(pattern, "abc")    # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater {} define um número específico de repetições
pattern = r"ab{2,4}c"
result = re.match(pattern, "abbc")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 4),
match='abbc'>
result = re.match(pattern, "abbbc")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 5),
match='abbbc'>
result = re.match(pattern, "abbbbbc")    # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater \d corresponde a qualquer dígito
(equivalente a [0-9])
pattern = r"\d"
result = re.match(pattern, "3")    # Corresponde
print(result)    # Saída: <re.Match object; span=(0, 1),
match='3'>
result = re.match(pattern, "a")    # Não corresponde
print(result)    # Saída: None

# -----
# Metacarater \w corresponde a qualquer caractere de
palavra (letras, números e _)
```

```
pattern = r"\w"
result = re.match(pattern, "a") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 1),
match='a'>
result = re.match(pattern, "3") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 1),
match='3'>
result = re.match(pattern, "#") # Não corresponde
print(result) # Saída: None

# -----
# Metacarater \s corresponde a qualquer caractere de
espaço em branco
pattern = r"\s"
result = re.match(pattern, " ") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 1),
match=' '>
result = re.match(pattern, "\t") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 1),
match='\t'>
result = re.match(pattern, "a") # Não corresponde
print(result) # Saída: None

# -----
# Metacarater \b corresponde a uma fronteira de palavra
pattern = r"\bword\b"
result = re.match(pattern, "word123") # Não corresponde
print(result) # Saída: None
result = re.match(pattern, " word ") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 4),
match='word'>

# -----
# Metacarater \B corresponde a uma não fronteira de
palavra
pattern = r"\Bword\B"
result = re.match(pattern, "word123") # Corresponde
print(result) # Saída: <re.Match object; span=(0, 4),
match='word'>
result = re.match(pattern, " word ") # Não corresponde
print(result) # Saída: None

# -----
# Grupos de captura e modificadores (?...)
pattern = r"(?i)abc"
result = re.match(pattern, "ABC") # Corresponde (ignora
maiúsculas/minúsculas)
print(result) # Saída: <re.Match object; span=(0, 3),
match='ABC'>
```

No contexto de NLP, as expressões regulares são utilizadas para realizar tarefas que envolvem estruturas simples e bem definidas, como:

- **Limpeza e pré-processamento de texto:** remover pontuações, números, URLs, símbolos gráficos como emojis etc.
- **Tokenização:** separar palavras ou frases com base em padrões simples.
- **Extração de entidades:** encontrar padrões específicos como endereços de e-mail, datas, números de telefone etc.

Vejamos um exemplo de Regex aplicado na extração de URLs em strings no código-fonte “Extração de URLs usando Regex em Python”. Nele, definimos uma função `find_url` que recebe uma string. Dentro desta função há um padrão de expressão regular definido na variável `url_regex`. As URLs são encontradas pela função `findall` que procura o padrão contido em `url_regex`.

```
import re

def find_url(string):
    url_regex = r'(?i)\b(?:https?|(?:{1,3}|[a-z0-9%])|
[a-z0-9.\-]+[.](?:com|net|org|edu|gov|mil|aero|asia|biz|cat|coop|info|
int|jobs|mobi|museum|name|post|pro|tel|travel|xxx|ac|ad|ae|af|
ag|ai|al|am|an|ao|aq|ar|as|at|au|aw|ax|az|ba|bb|bd|be|bf|bg|b
h|bi|bj|bm|bn|bo|br|bs|bt|bv|bw|by|bz|ca|cc|cd|cf|cg|ch|ci|ck
|cl|cm|cn|co|cr|cs|cu|cv|cx|cy|cz|dd|de|dj|dk|dm|do|dz|ec|ee|
eg|eh|er|es|et|eu|fi|fj|fk|fm|fo|fr|ga|gb|gd|ge|gf|gg|gh|gi|g
l|gm|gn|gp|gq|gr|gs|gt|gu|gw|gy|hk|hm|hn|hr|ht|hu|id|ie|il|im
|in|io|iqr|is|it|je|jm|jo|jp|ke|kg|kh|ki|km|kn|kp|kr|kw|ky|
kz|la|lb|lc|li|lk|lr|ls|lt|lu|lv|ly|ma|mc|md|me|mg|mh|mk|ml|m
m|mn|mo|mp|mq|mr|ms|mt|mu|mv|mw|mx|my|mz|na|nc|ne|nf|ng|ni|nl
|no|np|nr|nu|nz|om|pa|pe|pf|pg|ph|pk|pl|pm|pn|pr|ps|pt|pw|py|
qa|re|ro|rs|ru|rw|sa|sb|sc|sd|se|sg|sh|si|sj|Ja|sk|sl|sm|sn|s
o|sr|ss|st|su|sv|sx|sy|sz|tc|td|tf|tg|th|tj|tk|tl|tm|tn|to|tp
|tr|tt|tv|tw|tz|ua|ug|uk|us|uy|uz|va|vc|ve|vg|vi|vn|vu|wf|ws|
ye|yt|yu|za|zm|zw)/)(?:[^\s()<>{}|\[\]]+|\([^\s()]*?\([^\s()]+
\)[^\s()]*?\)|\[([^\s]+?)\])+(?:\:|([^\s()]*?\([^\s()]+\)[^\s()]*
?)|\[([^\s]+?)\]|([^\s`!() \[\]{};:'".,<>?«»"'"`])|(?:(?![@]
[a-z0-9]+(?:[. \-][a-z0-9])*)*\.)(?:com|net|org|edu|gov|mil|aero|asia|biz|cat|coop|inf
o|int|jobs|mobi|museum|name|post|pro|tel|travel|xxx|ac|ad|ae|
af|ag|ai|al|am|an|ao|aq|ar|as|at|au|aw|ax|az|ba|bb|bd|be|bf|b
g|bh|bi|bj|bm|bn|bo|br|bs|bt|bv|bw|by|bz|ca|cc|cd|cf|cg|ch|ci
|ck|cl|cm|cn|co|cr|cs|cu|cv|cx|cy|cz|dd|de|dj|dk|dm|do|dz|ec|
ee|eg|eh|er|es|et|eu|fi|fj|fk|fm|fo|fr|ga|gb|gd|ge|gf|gg|gh|g
i|gl|gm|gn|gp|gq|gr|gs|gt|gu|gw|gy|hk|hm|hn|hr|ht|hu|id|ie|il
```

```
|im|in|io|iql|ir|is|it|je|jm|jo|jp|ke|kg|kh|ki|km|kn|kp|kr|kw|
ky|kz|la|lb|lc|li|lk|lr|ls|lt|lu|lv|ly|ma|mc|md|me|mg|mh|mk|m
l|mm|mn|mo|mp|mq|mr|ms|mt|mu|mv|mw|mx|my|mz|na|nc|ne|nf|ng|ni
|nl|no|np|nr|nu|nz|om|pa|pe|pf|pg|ph|pk|pl|pm|pn|pr|ps|pt|pw|
py|qa|re|ro|rs|ru|rw|sa|sb|sc|sd|se|sg|sh|si|sj|Ja|sk|sl|sm|s
n|so|sr|ss|st|su|sv|sx|sy|sz|tc|td|tf|tg|th|tj|tk|tl|tm|tn|to
|tp|tr|tt|tv|tw|tz|ua|ug|uk|us|uy|uz|va|vc|ve|vg|vi|vn|vu|wf|
ws|ye|yt|yu|za|zm|zw)\b/(?(!@)) ) ""
    url = re.findall(url_regex, string)
    return (url)

string_url      =      "entre      nesse      link      aqui
https://g1.globo.com/"
urls = find_url(string_url)
print(urls)
```

Código-fonte 2 - Extração de URLs usando Regex em Python
Fonte: Elaborado pelo autor (2025)

Por que o padrão a ser procurado pela função `findall` é tão grande? Pois precisamos escrever todas as formas possíveis para uma URL. Assim, padrões regex podem ser extremamente longos e cansativos de se fazer. Eles constituem a primeira ferramenta em NLP e ainda são frequentemente aplicados (por exemplo, para implementar a operação de Ctrl+F). Apesar de sua grande utilidade, expressões regulares não são adequadas para lidar com estruturas linguísticas mais complexas, como a **análise sintática** profunda de frases, que envolve dependências hierárquicas e contextuais — como a análise de frases compostas e ambíguas — e tarefas que exigem **gramáticas livres de contexto** ou **modelos baseados em Aprendizado de Máquina**.

5 ANÁLISE LEXICAL

5.1 Analisadores Léxicos em Compiladores

O analisador léxico (também chamado de lexer ou scanner) é utilizado como uma das primeiras fases de um processo de compilação de código. Ele é responsável por **ler o código-fonte** (geralmente em uma linguagem de programação) e dividi-lo em unidades significativas chamadas **tokens**. Tokens são as menores unidades da linguagem que têm um significado específico, como palavras-chave, identificadores, operadores, literais e pontuação.

O analisador léxico executa os seguintes passos:

- **Passo 1 – Leitura do código-fonte:** o analisador léxico percorre o código-fonte caractere por caractere.
- **Passo 2 – Identificação de tokens:** ele identifica os diferentes tokens de acordo com a gramática da linguagem. Cada token pode ser uma palavra-chave, identificador, operador, número etc.
- **Passo 3 – Remoção de espaços em branco e comentários:** espaços em branco, quebras de linha e comentários são ignorados, pois não têm relevância para a análise sintática ou semântica na compilação.
- **Passo 4 – Geração de tokens:** após identificar um token, o analisador léxico gera um objeto que representa esse token. Esse objeto normalmente contém informações como o tipo de token e a posição do texto no código-fonte.

Os tokens de uma análise léxica utilizada em processos de compilação possuem tipos bem definidos:

- **Palavras-chave (keywords):** são palavras reservadas da linguagem, como `if`, `for`, `while`, `int`, `return`.
- **Identificadores:** nomes dados a variáveis, funções ou outros elementos definidos pelo programador, como, `x` e `minhaFuncao`.
- **Literais:** constantes no código, como números (10, 5) e strings ("x é maior que 5").

- **Operadores:** símbolos que representam operações, como `+`, `-`, `=`, `>`.
- **Delimitadores:** caracteres usados para estruturar o código, como o ponto e vírgula `;`, parênteses `( )`, chaves `{ }`, a indentação com marcador tabulação `\t`, e a separação de instruções pela quebra de linha `\n`.
- **Comentários:** embora não influenciem a execução do programa, são importantes para a documentação. O analisador léxico os ignora. Exemplo: a sequência de caracteres `//` que delimita o início de um comentário em C, e o caractere `#` que delimita o início de comentário em Python.

A implementação do analisador léxico pode ser realizada de várias formas, mas a abordagem mais comum é utilizar expressões regulares para identificar padrões de tokens. O analisador léxico prepara o caminho para a próxima fase do compilador — o **analisador sintático** (parser), responsável por verificar se a sequência de tokens obedece à gramática da linguagem.

5.2 Analisadores Léxicos em NLP

Da mesma forma que os lexers são utilizados em compiladores para analisar e segmentar um texto em unidades menores (os tokens), eles também são empregados em NLP para quebrar sentenças e documentos em palavras, símbolos, números ou outros elementos com significado dentro de um contexto linguístico.

O papel do analisador léxico é essencial na construção de sistemas de NLP, pois ele ajuda a transformar um fluxo contínuo de texto em estruturas que podem ser processadas por etapas posteriores, como a análise sintática e a análise semântica. Vamos entender um pouco mais sobre as funções realizadas por analisadores léxicos em NLP:

- **Segmentação de Texto (Tokenização):** a principal função do analisador léxico é dividir o texto em partes menores chamadas tokens. Por exemplo, a frase “Eu gosto de programar” seria segmentada nos tokens: “Eu”, “gosto”, “de”, “programar”.
- **Classificação de Tokens:** cada token identificado é classificado em uma categoria gramatical ou tipo (também chamados de **tags**), como substantivo, verbo, adjetivo, numeral etc.

- **Remoção de Tokens Não Relevantes (Stopwords):** durante a tokenização, muitas vezes é importante remover palavras irrelevantes (como artigos e preposições) que não contribuem significativamente para o significado do texto em tarefas específicas.
- **Identificação de Entidades Nomeadas:** em algumas implementações de analisadores léxicos, pode haver o reconhecimento de entidades nomeadas, como pessoas, lugares, datas etc.

Uma semelhança interessante entre os analisadores léxicos de código-fonte e os analisadores léxicos de linguagem natural é que ambos são dependentes da língua (instância de linguagem) a ser analisada. Por exemplo, em C++ e Python os tokens de palavra-chave podem ser diferentes, assim como os tokens para verbos são diferentes em inglês e português. Isso faz com que o analisador léxico seja construído aproximadamente de acordo com a linguagem que ele irá analisar. Por outro lado, enquanto em lexers de compiladores os tipos de tokens são bem definidos e comportados, os tokens de lexers de linguagem natural são muito variados: temos verbo, substantivo, adjetivo, advérbio, pronome, palavras no singular e no plural, entre vários outros tipos de classes gramaticais.

De maneira geral, existem duas formas de se construir um analisador léxico:

- **Analisadores baseados em regras:** utilizam regras explícitas para identificar padrões e segmentar o texto. Por exemplo, pode-se usar uma regra para identificar palavras que terminam com uma certa sequência de caracteres ou para identificar pontuações e separar frases.
- **Analisadores baseados em aprendizado de máquina:** usam algoritmos de Aprendizado de Máquina (como Redes Neurais) para classificar palavras e identificar tokens. Esses analisadores aprendem a partir de grandes quantidades de dados.

Existem várias bibliotecas que implementam funções de NLP para realizar a análise léxica. Uma das mais populares é o **NLTK (Natural Language Toolkit)**, uma biblioteca em Python amplamente utilizada para o Processamento de Linguagem Natural. O NLTK oferece uma vasta gama de ferramentas, incluindo funções para tokenização, análise sintática, stemming, lematização, etiquetagem (tagging) de partes do discurso e análise de sentimentos. Além disso, o NLTK inclui recursos como

corpora e lexicons, que são conjuntos de dados pré-processados utilizados para treinar modelos e realizar experimentos.

Vamos começar com um exemplo simples de uso do tokenizador em português presente no NLTK. Primeiro, devemos instalar a biblioteca. Você pode fazer isso na sua instalação local do Python através do comando:

```
pip install nltk
```

Comando de prompt 1 - Instalação do NLTK
Fonte: Elaborado pelo autor (2025)

Caso esteja utilizando um ambiente desenvolvido no notebook como o plugin do VSCode, o Google Colab ou a instalação local Jupyter Notebook, você pode executar esse mesmo comando em uma célula de código dentro do notebook .ipynb adicionando ! (exclamação) na frente do código.

O NLTK **Punkt** é um modelo de tokenização que pode ser utilizado para segmentar texto em sentenças e palavras, e que possui suporte para vários idiomas, incluindo o português. Esse sistema de tokenização é baseado em Aprendizado de Máquina, de forma que não depende de regras fixas para realizar a segmentação, mas sim de um modelo probabilístico que aprende padrões a partir de dados de texto. Vamos baixar o modelo do punkt. Dentro do seu código Python, execute:

```
nltk.download('punkt')
```

Código-fonte 3 - Código Python para o download do modelo de tokenização do NLTK
Fonte: Elaborado pelo autor (2025)

Agora podemos utilizar o tokenizador:

```
from nltk.tokenize import word_tokenize

texto = "Olá! Como você está? Vamos aprender sobre análise léxica com NLTK."

tokens = word_tokenize(texto, language='portuguese')
print("Tokens:", tokens)
```

Código-fonte 4 - Demonstração do uso do analisador léxico do NLTK em Python
Fonte: Elaborado pelo autor (2025)

Note que o simples comando Python `texto.split(" ")` não seria suficiente para retornar a lista que, inclusive, separa a pontuação gráfica. Isso porque o `split` apenas quebra nos caracteres passados no argumento da função, enquanto o `word_tokenize` de fato utiliza conhecimento acerca do vocabulário da língua portuguesa para quebrar a sentença em palavras.

Dica: você pode saber mais sobre o NLTK em: <<https://www.nltk.org/>>.

Outra função presente no NLTK referente ao vocabulário são as listas de **Stopwords**. Esses tokens são considerados de baixa relevância semântica, ou seja, são palavras que não contribuem de maneira significativa para o entendimento geral de uma frase. É comum que para determinadas tarefas como classificação de intenções e análise de sentimento, o texto seja filtrado para que essas palavras sejam removidas. Vejamos como fazer isso:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Estamos baixando o corpus contendo as stopwords
nltk.download('stopwords')

texto = "Este é um exemplo de texto onde vamos remover as
palavras stopwords."

# Tokenizar o texto (dividir o texto em palavras)
palavras = word_tokenize(texto, language='portuguese')

# Obter a lista de stopwords em português
stop_words = set(stopwords.words('portuguese'))

# Remover as stopwords
palavras_filtradas = [palavra for palavra in palavras if
palavra.lower() not in stop_words]

# Resultado
texto_sem_stopwords = ' '.join(palavras_filtradas)
print("Frase original: ", texto)
print("Nova frase: ", texto_sem_stopwords)
```

Código-fonte 5 - Remoção de stopwords usando NLTK em Python

Fonte: Elaborado pelo autor (2025)

Caso você já tenha baixado o módulo de stopwords, você pode ignorar essa linha do código. Observe que, ao utilizar o código-fonte “Remoção de stopwords usando NLTK em Python” para descobrir quais são as palavras consideradas stopwords em português, você apenas precisará printar a variável `stop_words` fazendo `print(stop_words)`.

Outra biblioteca amplamente utilizada para Processamento de Linguagem Natural é o spaCy, que se destaca pela sua eficiência e desempenho em tarefas de NLP em grande escala. Diferente do NLTK, que é mais focado em ensino e pesquisa,

o spaCy é projetado para ser rápido e robusto, sendo ideal para aplicações em produção. Ele oferece funcionalidades avançadas, como Análise Sintática Profunda, Reconhecimento de Entidades Nomeadas (NER), Lematização, e Treinamento de Modelos Personalizados, tudo isso com uma API simples e de fácil integração. Além disso, o spaCy é altamente otimizado para trabalhar com grandes volumes de dados, tornando-o uma escolha popular em ambientes industriais e comerciais.

Vejamos como repetir esses mesmos exemplos feitos em NLTK usando o spaCy. Primeiro a instalação:

```
pip install spacy
```

Comando de prompt 2 - Instalação do spaCy
Fonte: Elaborado pelo autor (2025)

Para usar o spaCy, é necessário instalar o módulo que contém os modelos em português. Existem três modelos para o idioma, todos criados com base em texto escrito de mídias e notícias. Sendo eles: `pt_core_news_sm` com 12 MB de tamanho; `pt_core_news_md` com 40 MB; e `pt_core_news_lg` com 521 MB de tamanho. Todos possuem analisador léxico, além de outras ferramentas de NLP como analisador sintático, lematizador, sentenciador (senter) e NER.

```
python -m spacy download pt_core_news_sm
```

Comando de prompt 3 - Instalação do módulo de português do spaCy
Fonte: Elaborado pelo autor (2025)

Novamente, você pode executar ambos os comandos no ambiente do Notebook adicionando a interrogação (!) no começo do código em uma célula de código.

Para o teste do analisador sintático do spaCy:

NLP e Chatbots: Vozes Digitais que Compreendem

```
import spacy

# Carregar o modelo de português
nlp = spacy.load("pt_core_news_sm")

# Texto a ser tokenizado
texto = "Olá! Como você está? Vamos aprender sobre análise léxica com spaCy."

# Processar o texto com o spaCy
doc = nlp(texto)

# Exibir os tokens
tokens = [token.text for token in doc]
print("Tokens:", tokens)
```

Código-fonte 6 - Demonstração do uso do analisador léxico do spaCy em Python
Fonte: Elaborado pelo autor (2025)

Já para realizar a remoção de stopwords em spaCy seria:

```
import spacy

# Carregar o modelo de linguagem em português do spaCy
nlp = spacy.load("pt_core_news_sm")

# Texto de exemplo
texto = "Este é um exemplo de texto onde vamos remover as palavras stopwords."

# Processar o texto
doc = nlp(texto)

# Remover as stopwords
palavras_filtradas = [token.text for token in doc if not token.is_stop]

# Resultado
texto_sem_stopwords = ' '.join(palavras_filtradas)
print("Frase original: ", texto)
print("Nova frase: ", texto_sem_stopwords)
```

Código-fonte 7 - Remoção de stopwords usando spaCy em Python
Fonte: Elaborado pelo autor (2025)

Note que o resultado das frases filtradas com NLTK e spaCy são diferentes. Isso ocorre, pois, cada uma das ferramentas possui modelos de português diferentes.

Dica: você pode saber mais sobre o spaCy em: <<https://spacy.io/>>.

Analísadores léxicos normalmente têm dificuldades com ambiguidades causadas por polissemia — quando uma palavra possui múltiplos significados —, o que pode dificultar a segmentação e a classificação. Por exemplo, a palavra "banco" pode se referir a uma instituição financeira ou a um tipo de móvel. Diferentes formas de grafar determinadas palavras também podem prejudicar a etapa de tokenização. Por exemplo, no contexto de química, a fórmula da molécula de água pode ser grafada como H2O, H2-O ou H₂O, cada forma possuindo caracteres diferentes que podem acabar gerando interpretações diferentes do analisador. Palavras compostas podem ter o mesmo tipo de problema como “d’água” que um determinado analisador léxico pode entender como um único token “d’água”, enquanto outro analisador entenderá como três tokens “d”, “ ’ ”, “água”. Outro problema decorre da constante evolução da linguagem, com novas palavras, gírias, formas de escrita e neologismos. Estas mudanças na língua natural exigem que os analisadores léxicos sejam atualizados ou treinados com dados recentes.

5.3 Lei de Zipf

A Lei de Zipf, descoberta pelo filólogo e linguista George Zipf, é um princípio estatístico que descreve a distribuição de frequências das palavras em um corpo de texto. Essa lei sugere que, em um grande conjunto de dados linguísticos, a frequência das palavras segue uma distribuição inversamente proporcional à sua posição em uma lista ordenada pela frequência. Em outras palavras, a palavra mais frequente aparece aproximadamente o dobro de vezes que a segunda mais frequente, três vezes mais que a terceira, e assim por diante. Essa lei revela que uma pequena quantidade de palavras é responsável pela maior parte do uso no idioma, enquanto a grande maioria das palavras ocorre com baixa frequência. A Lei de Zipf tem implicações significativas na linguística e em áreas como Processamento de Linguagem Natural e análise de texto, pois ela ajuda a entender como as palavras são distribuídas em corpora e como isso pode ser modelado para melhorar a eficiência de algoritmos de NLP.

Vamos observar a Lei de Zipf na prática usando NLTK. Primeiro, precisamos de um corpus de texto para trabalharmos em cima. O **Projeto Gutenberg** é uma plataforma web que distribui milhares de livros em domínio público. Ele pode ser acessado em: <<https://www.gutenberg.org/>>. Analisaremos o romance Quincas

NLP e Chatbots: Vozes Digitais que Compreendem

Borba, de Machado de Assis (<<https://www.gutenberg.org/ebooks/55682>>). Entre no link correspondente e faça o download do arquivo de texto puro (*plain text*). Ao abrir o link, o texto será aberto no navegador, clique com botão direito do mouse e escolha a opção “salvar como” (em inglês, “save as”). Na tela de salvamento, salve como **.txt**. A seguir, a figura apresenta uma sequência de capturas de tela desse procedimento. Em seguida, coloque o arquivo baixado na mesma pasta em que estiver rodando seu código em Python. Caso esteja utilizando um ambiente em nuvem como o Google Colab, lembre-se de fazer o upload do arquivo para a máquina virtual no menu lateral esquerdo.

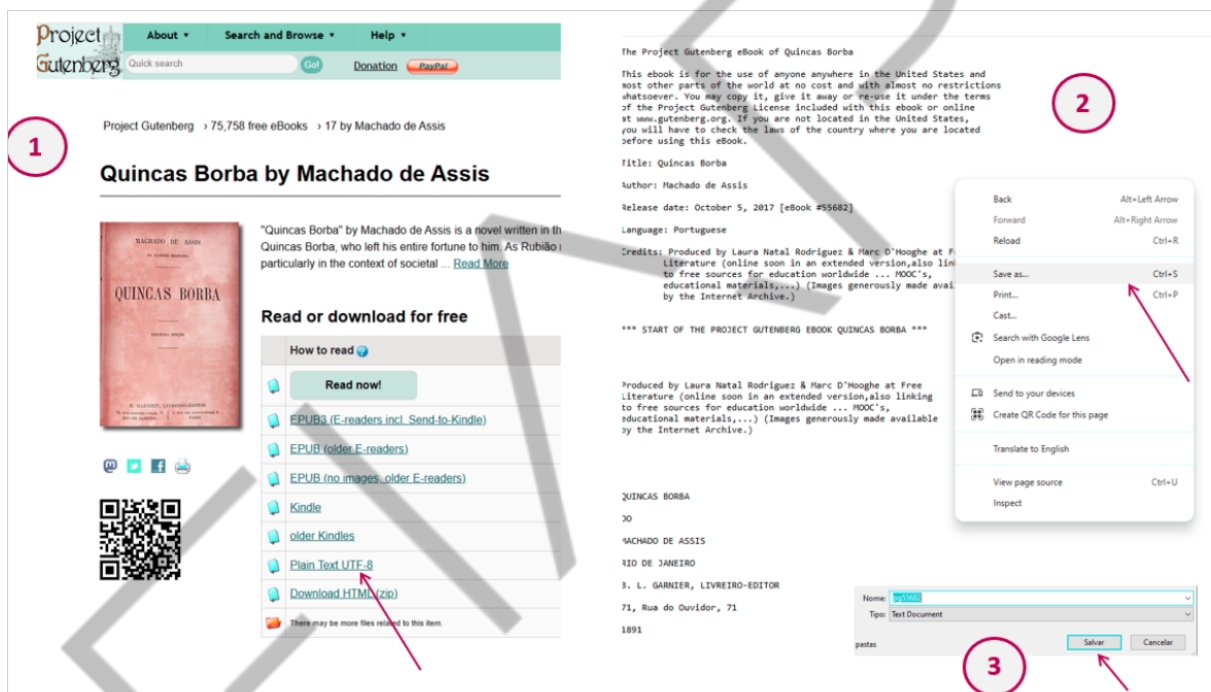


Figura 6 - Sequência de telas para o download do corpus de texto do Projeto Gutenberg
Fonte: Elaborado pelo autor (2025)

O código para carregar o arquivo e analisar a frequência de ocorrência dos tokens é:

```
import nltk
import matplotlib.pyplot as plt
from nltk.probability import FreqDist
from nltk.tokenize import word_tokenize
import numpy as np

# Carregar o texto do arquivo
with open('pg55682.txt', 'r', encoding='utf-8') as file:
    text = file.read()

# Tokenizar o texto em palavras
```

```
words = word_tokenize(text)

# Criar a distribuição de frequência das palavras
fdist = FreqDist(words)

# Ordenar as palavras por frequência (da mais frequente
para a menos frequente)
sorted_words = sorted(fdist.items(), key=lambda x: x[1],
reverse=True)

# Obter as frequências e as posições das palavras
frequencies = [freq for word, freq in sorted_words]
ranks = np.arange(1, len(frequencies) + 1)

# Plotar o gráfico
plt.figure(figsize=(8, 4))
plt.plot(ranks, frequencies, marker='o', linestyle='-',
color='b')
plt.xscale('log') # Escala logarítmica no eixo x (ranks)
plt.yscale('log') # Escala logarítmica no eixo y
(frequências)
plt.xlabel('Rank (log)')
plt.ylabel('Frequência (log)')
plt.title('Lei de Zipf - Distribuição de Frequência das
Palavras')
plt.grid(True)

# Anotar as três primeiras palavras com setas
for i, (word, freq) in enumerate(sorted_words[:3]):
    plt.annotate(f'{word} ({freq})',
                xy=(i+1, freq),
                xytext=(i+1+5, freq),
                arrowprops=dict(arrowstyle='->',
lw=0.5),
                horizontalalignment='left')

# Anotar a 100ª palavra com seta, se houver
if len(sorted_words) >= 100:
    word, freq = sorted_words[99]
    plt.annotate(f'{word} ({freq})',
                xy=(100, freq),
                xytext=(100+50, freq),
                arrowprops=dict(arrowstyle='->',
lw=0.5),
                horizontalalignment='left')

plt.show()
```

Código-fonte 8 - Análise de frequência dos tokens presentes em Quincas Borba

Fonte: Elaborado pelo autor (2025)

Além de analisar a frequência, o código-fonte “Análise de frequência dos tokens presentes em Quincas Borba” está plotando um gráfico contendo a frequência (eixo y) e o ranking (eixo x) de cada palavra. Note que o token mais presente (1ª no ranking) em Quincas Borba é a vírgula (,) que aparece 8757 vezes. A palavra Borba é exatamente a 100ª palavra mais dita no texto, aparecendo 97 vezes. O resultado da execução desse código é ilustrado na figura “Demonstração de Lei de Zipf em Quincas Borba de Machado de Assis. Esse mesmo gráfico é gerado ao ser executado o código-fonte anterior”. Cada ponto representa uma palavra (token). Observe que há poucas palavras que aparecem muitas vezes e muitas palavras que aparecem pouquíssimas vezes. Essa é exatamente a Lei de Zipf e ela é esperada de ocorrer em qualquer léxico de linguagem natural. Inclusive, é uma forma de saber que determinado conjunto de lexemas (palavras) constitui de fato uma linguagem natural!

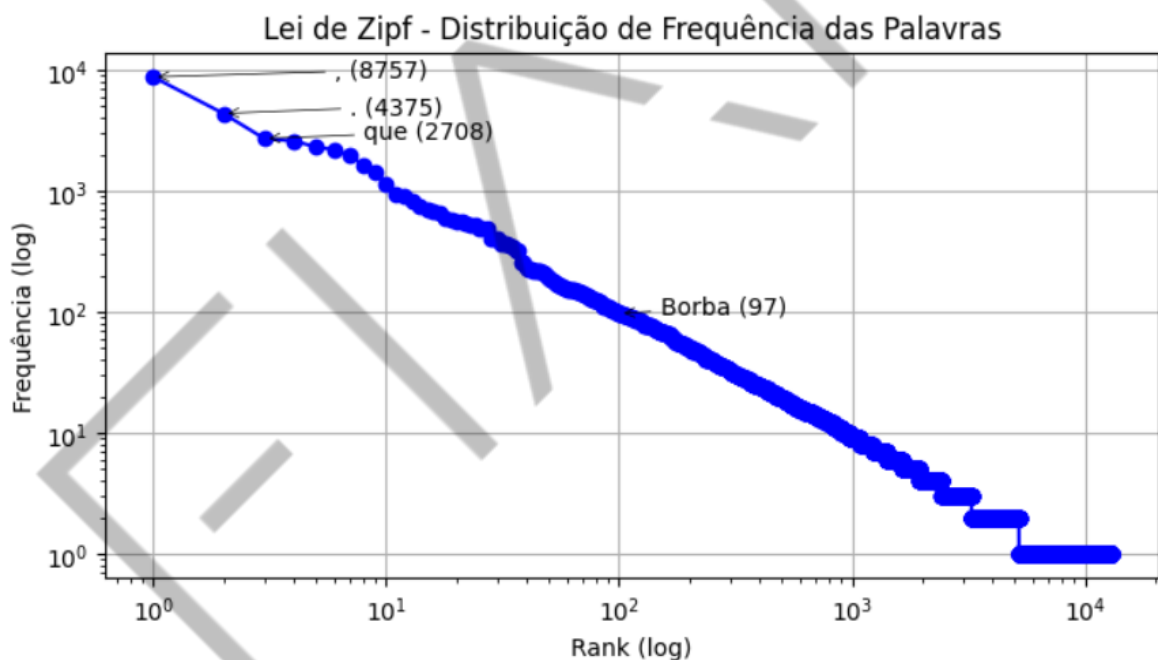


Figura 7 - Demonstração de Lei de Zipf em Quincas Borba de Machado de Assis. Esse mesmo gráfico é gerado ao ser executado o código-fonte anterior

Fonte: Elaborado pelo autor (2025)

REFERÊNCIAS

BOMMASANI, Rishi. *et al.* On the Opportunities and Risks of Foundation Models. **arXiv**, 16 de agosto de 2021. Disponível em: <<https://arxiv.org/abs/2108.07258>>. Acesso em: 16 maio 2025.

DOMINGOS, Pedro. **O Algoritmo Mestre**: como a busca pelo algoritmo de machine learning definitivo recriará nosso mundo. 1. ed. São Paulo: Novatec, 2017.

GUO, Daya. *et al.* DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. **arXiv**, 22 de janeiro de 2025. Disponível em: <<https://arxiv.org/abs/2501.12948>>. Acesso em: 16 maio 2025.

JURAFSKY, Dan; MARTIN, James. **Speech and Language Processing**. 3. ed. Stanford, 2025. Disponível em: <<https://web.stanford.edu/~jurafsky/slp3/>>. Acesso em: 16 maio 2025.

MIKOLOV, Tomas. *et al.* Efficient Estimation of Word Representations in Vector Space. **arXiv**, 16 de janeiro de 2013. Disponível em: <<https://arxiv.org/abs/1301.3781>>. Acesso em: 16 maio 2025.

NORVIG, Peter; RUSSEL, Stuart. **Inteligência Artificial**. 3. ed. Rio de Janeiro: GEN LTC, 2013.

RAJ, Sumit. **Construindo Chatbots com Python**: Usando Natural Language Processing e Machine Learning. 1. ed. São Paulo: Novatec, 2019.

RICH, Elaine. **Inteligência Artificial**. 1. ed. São Paulo: McGraw-Hill, 1988.

SIPSER, Michael. **Introdução à Teoria da Computação**. 1. ed. São Paulo: Cengage Learning, 2005.

TURING, Alan. **Computer Machinery and Intelligence**. *Mind* 49, p. 433-460, 1950.

VASWANI, Ashish. *et al.* Attention Is All You Need. **arXiv**, 12 de junho de 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Acesso em: 16 maio 2025.

WEIZENBAUM, Joseph. ELIZA — a computer program for the study of natural language communication between man and machine. **Communications of the ACM**, Volume 9, Issue 1, p. 36-45, 1966.

WINSTON, Patrick. **Inteligência Artificial**. 1. ed. Rio de Janeiro: LTC, 1988.